

Stabilizing In-Context Multi-Source Domain Adaptation for Biomedical Images Through Controls

Ana Sanchez-Fernandez¹ Thomas Pinetz² Werner Zellinger¹ Günter Klambauer^{1,3}

Abstract

Biomedical imaging data presents enormous potential for deep learning models to predict invaluable properties, such as diseases and drug effects. However, unavoidable alterations of the technical conditions cause *batch effects*: variations between groups of samples that are not due to any biological signal of interest. Batch effects greatly hinder the generalization abilities of deep learning models, preventing their practical use in the real world. Unsupervised Domain Adaptation (UDA) methods have been proposed to mitigate batch effects, but they usually assume that the data is comprised of only one source domain and one target domain, whereas biological datasets are comprised of multiple domains, both at training and at inference time. While Batch Normalization-based test-time and meta-learning adaptation methods offer a promising mechanism for domain alignment, we show that existing approaches exhibit degraded performance under the usual inference scenarios of small target batch sizes and label shift. We address these limitations by leveraging negative control samples, which are consistently present in every experimental batch in biological datasets, as stable context for adaptation. We propose CS-ARM-BN, a meta-learning BN adaptation method that uses controls both during training and inference to stabilize domain statistics. We perform a suite of experiments of Mechanism-Of-Action (MoA) classification, a crucial task for drug discovery, on the large JUMP-CP imaging dataset. Our experiments show that CS-ARM-BN substantially improves robustness to batch size and class distribution shifts, enabling practical use of deep learning models for biomedical images.

¹ELLIS Unit Linz, LIT AI Lab and Institute for Machine Learning, JKU Linz, Austria ²Institute of Artificial Intelligence, Center for Medical Data Science, Medical University Vienna ³Clinical Research Center for Medical AI, JKU Linz, Austria. Correspondence to: Günter Klambauer <klambauer@ml.jku.at>.

Proceedings of the 43rd International Conference on Machine Learning, Seoul, South Korea. PMLR 306, 2026. Copyright 2026 by the author(s).

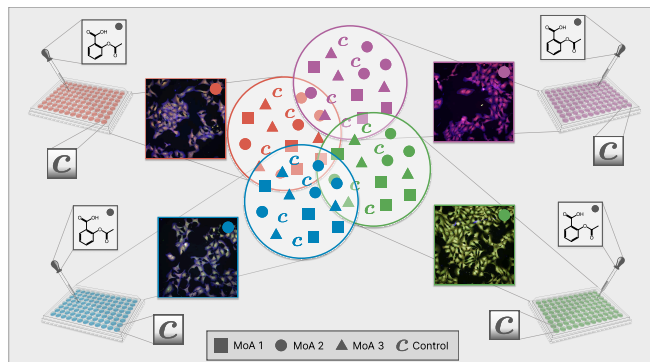


Figure 1. Representation of *batch effects* in microscopy imaging data considered as a multi-source domain adaptation (MSDA) problem. In this setting, each source consists of different experimental conditions, e.g. different plates. In each domain, an image of the same class is depicted, which represents a particular mechanism-of-action (MoA). Control samples are unperturbed samples, that are present in every domain, and which we represent with the *c* symbol.

1. Introduction

Batch effects severely degrade model generalization for biomedical datasets. The growing scale of publicly available biomedical imaging datasets offers an unprecedented opportunity for deep learning models to predict clinically and biologically meaningful properties, such as disease states, treatment responses, and drug effects (Chandrasekaran et al., 2023; Zhang et al., 2025; Peidli et al., 2023). However, a key challenge arises because biological data has to be acquired in *experimental batches*, which are groups of samples that were obtained under different technical conditions. Even in carefully controlled environments, images from different batches present variations due to non-biological factors that might be more prominent than the biological signal in question (Leek et al., 2010; Syetkowski et al., 2023; Luecken et al., 2021). These variations are called *batch effects* and are a well-known artifact, such that unperturbed samples, known as *negative controls*, are included in every experimental batch as a reference (Bray et al., 2017). Negative controls have been long used in the field of cellular imaging profiling to try to disentangle biological signal from confounding factors, often by stan-

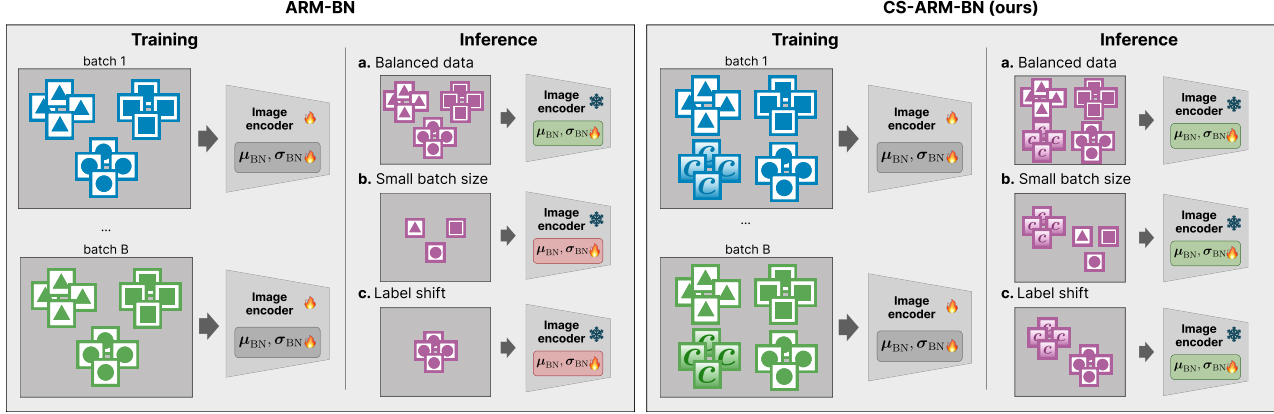


Figure 2. Graphic representation of our method, CS-ARM-BN, and comparison to ARM-BN (Zhang et al., 2021). Both are meta-learning methods that are modified at test-time by using the BN statistics from the target domain (lilac). CS-ARM-BN uses control samples both at training and at inference time, which provides stability when (b) the number of perturbed samples is small or (c) the label distribution is shifted.

standardizing features using controls from the corresponding experimental batch (Arevalo et al., 2024). However, such post-hoc corrections do not allow exploiting the capabilities of end-to-end deep learning models, and traditional deep learning models perform poorly on new batches (Chen et al., 2020; Kim et al., 2025).

Domain adaptation provides a natural framework for batch effect correction. We could consider batch effect correction in machine learning as an *unsupervised domain adaptation* (UDA) problem, where the source domain is the training data available and the target domain is a new experimental batch. Classical UDA aligns source and target distributions using approaches such as a) importance weighting (Shimodaira, 2000; Ben-David et al., 2006; Kimura and Hino, 2024), b) distance-based matching (Sun et al., 2015; Zellinger et al., 2017), c) adversarial training (Ganin and Lempitsky, 2014), or d) generative translation (Liu and Tuzel, 2016; Taigman et al., 2016; Bousmalis et al., 2017; Chung et al., 2024). In the field of microscopy imaging data, test-time training (TTT) strategies have been explored (Haslum et al., 2023). However, traditional UDA and TTT methods assume a small number of source domains, each of them with a large amount of samples, and require retraining the model every time a new domain needs to be evaluated. In contrast, microscopy imaging data is comprised of many domains with few samples (Chandrasekaran et al., 2023; Bray et al., 2017), which makes UDA methods unsuitable in this setting. Batch effect correction can be formalized instead as a *multi-source domain adaptation* (MSDA) problem (see Figure 1), where each experimental batch is a source domain (Farahani et al., 2020; Blanchard et al., 2011).

Multi-source domain adaptation and in-context test-time adaptation. Recent works have been developed to extend classical UDA methods to a multi-source setting (Zhao et al.,

2018; Peng et al., 2019; Wen et al., 2020; Yang et al., 2020; Venkat et al., 2020; Guo et al., 2018). Nevertheless, just like UDA, MSDA methods require re-training the model every time a new target domain needs to be evaluated. In microscopy imaging data, new experimental batches come in sequentially, making these methods highly inefficient. An ideal setting for microscopy imaging data would be one where the multi-source nature of the data can be exploited and also the model can be dynamically adapted at test time without need for retraining. Adaptive Risk Minimization (ARM) (Zhang et al., 2021) is a meta-learning framework (Hochreiter et al., 2001; Finn et al., 2017) that enables such rapid adaptation.

Batch Normalization as a domain adaptation mechanism. Batch Normalization (BatchNorm) is a widely established normalization technique, that allowed training deep neural networks with skip connections (Ioffe and Szegedy, 2015), and that can additionally be used as a module for domain adaptation (Li et al., 2016). Conventionally, when the assumption that both training and test data are sampled from the same distribution is made, the stored training statistics are used at inference time. In contrast, when the test samples are known to come from a different distribution, adapting these layers by using the statistics of the new domain at inference time provides a straight-forward test-time domain adaptation method (Li et al., 2016; Wang et al., 2021). However, it has also been shown that these methods fail when not enough samples are available to accurately estimate the new domain statistics or when the class distribution is shifted at test time (Zhao et al., 2023; Park et al., 2023).

Control samples provide an opportunity to stabilize the context in meta-learning methods. In addition to the methods presented above, there are also meta-learning methods which include adapting the batch normalization layers in a

meta-learning setting (Zhang et al., 2021). The main difference between BatchNorm adaptation (Li et al., 2016) and meta-learning BatchNorm adaptation (Zhang et al., 2021) is that the adaptation does not only occur at test-time but also during training, such that the model learns to adapt to new batches. This has been shown to provide a significant improvement with respect to its pure test-time counterparts. Nevertheless, in this work, we show that these methods also underperform in the presence of small batch sizes and under label shifts. Both scenarios, a) small sample sizes and b) label shifts in a new experimental batch are very plausible, because of the typical design of such bioimaging studies (Hughes et al., 2011; Knowles and Gromo, 2003). However, almost all biomedical experiments guarantee that a specific type of samples will be *present in every new experimental batch (i.e., domain): negative controls* (Chandrasekaran et al., 2023; Bray et al., 2017). Therefore, we propose to exploit the presence of negative controls and use them as context samples both during training and for inference of a meta-learning model, showing that this approach makes the performance of our method robust to small numbers of new experimental batches and to shifts in the class distribution.

Contributions. Our main contributions are the following:

- We propose **CS-ARM-BN**, a control-stabilized meta-learned BatchNorm adaptation method that leverages negative control samples available in every experimental batch, and show that it robustly alleviates the failure modes of previous methods and nearly closes the generalization gap between in-domain and unseen-batch performance.
- We demonstrate that adaptive Batch Normalization-based methods, particularly when combined with meta-learning and in-context adaptation, provide an effective and computationally efficient approach to mitigate severe batch effects in biomedical microscopy images.
- We systematically analyze and expose key limitations of existing BatchNorm-based adaptation methods, including AdaBN and ARM-BN, showing that their performance degrades in realistic settings such as label shift.

2. Problem setting and Background

We consider the problem of adapting from B source batches of size N to a single target batch of size M under the cross-entropy loss $\mathcal{L}_{\text{CE}}(\mathbf{y}, \hat{\mathbf{y}}) = -\sum_{k=1}^K y_k \log(\hat{y}_k)$. This problem can be extended to any loss function $\mathcal{L}$, many target batches and different batch sizes, in a straightforward way.

We are given a source dataset $\mathcal{D}^{\text{train}} = \{\mathcal{D}^{(1)}, \dots, \mathcal{D}^{(B)}\} \subset (\mathcal{X} \times \mathcal{Y})^N$ consisting of B experimental batches $\mathcal{D}^{(b)} = (\mathbf{x}_n^{(b)}, \mathbf{y}_n^{(b)})_{n=1}^N$ drawn from distributions $p_{x,y}^b$, $b \in \{1, \dots, B\}$, respectively. Follow-

ing Baxter (1998); Zhang et al. (2021), we assume that the batches are independent and drawn from a meta-distribution μ .

The goal is to find a prediction model $g(\mathbf{x}, \mathbf{w}) = \hat{\mathbf{y}}$ together with an adaptation method $\text{Adapt}(\mathbf{w}, g, \mathcal{D}^\beta) = \tilde{\mathbf{w}}$ that allow us, for any new *unlabeled* experimental target batch $\mathcal{D}^\beta = (\mathbf{x}_n^\beta)_{n=1}^M$, to adapt the weights $\mathbf{w}$ to new weights $\tilde{\mathbf{w}}$ with a small target batch risk

$$\frac{1}{M} \sum_{m=1}^M \ell(\mathbf{y}_m, g(\mathbf{x}_m, \tilde{\mathbf{w}})), \quad (1)$$

whenever the target batch is drawn from the marginal p_x^β of a new target distribution from μ . ℓ is an appropriate loss function.

Feature extractor and classifier networks. We denote the following components that are normally used by domain adaptation methods: a) *an encoder or feature extractor* network $\mathbf{f} : \mathcal{X} \mapsto \mathbb{R}^D$ or $\mathbf{f}_w$ with weights $\mathbf{w}$, which maps the input images $\mathbf{x} \in \mathcal{X}$ to an embedding $\mathbf{a} \in \mathbb{R}^D$, and b) *a classifier* $\mathbf{c} : \mathbb{R}^D \mapsto \mathcal{Y}$, which maps the embeddings to a class label $\hat{\mathbf{y}} \in \mathcal{Y}$ in the output space $\mathcal{Y}$.

The model $g : \mathcal{X} \mapsto \mathcal{Y}$ is usually a composition of the feature extractor with the classifier $g = \mathbf{c} \circ \mathbf{f} : \mathcal{X} \mapsto \mathcal{Y}$.

2.1. Domain Adaptation Methods for Biomedical Images

We first introduce the main domain adaptation methods used for comparison and referenced throughout the manuscript. See Supplementary Section D for descriptions of all compared methods.

Supervised baselines. ResNet (He et al., 2015) and Foundation Model CA-MAE (Kraus et al., 2024). We train a standard classifier g on $\mathcal{D}^{\text{train}}$ by minimizing

$$\min_{\mathbf{w}} \sum_{b=1}^B \sum_{(\mathbf{x}, \mathbf{y}) \in \mathcal{D}^{(b)}} \mathcal{L}_{\text{CE}}(\mathbf{y}, g(\mathbf{x}, \mathbf{w})) \quad (2)$$

with cross-entropy loss $\mathcal{L}_{\text{CE}}$ and using a ResNet50 architecture with either BatchNorm (BN) or InstanceNorm (IN) layers, and a linear layer as a classifier. This classifier is not adapted to new experimental batches.

For the foundation model baseline, we use a frozen CA-MAE encoder $\mathbf{f}_{\text{MAE}}$ to obtain embeddings $\mathbf{a} = \mathbf{f}_{\text{MAE}}(\mathbf{x})$ and perform linear probing with a classifier $\mathbf{c}_\theta$ trained with CE-loss.

Adaptive Batch Normalization (AdaBN) (Li et al., 2016). In Adaptive Batch Normalization, the adaptation of the model is simply carried out by using the BN statistics of the target domain at inference time, instead of those of the

training data. For microscopy images Lin and Lu (2022) proposed adapting the BN statistics based on control samples.

Fully Test-Time Adaptation by Entropy Minimization (TENT) (Wang et al., 2021). In TENT, the adaptation is also carried out at test-time, and, as in AdaBN, the BN statistics are computed using samples from the new domain. However, in this method, an additional adaptation is included, consisting of minimizing the entropy H of the model predictions $\hat{y}$ by updating only the BatchNorm affine parameters κ and λ :

$$\hat{y} = g(x), \quad H(\hat{y}) = - \sum_{k=1}^K \hat{y}_k \log(\hat{y}_k) \quad (3)$$

$$\min_{\kappa, \lambda} H(\hat{y}) \quad (4)$$

Adaptive Risk Minimization (ARM) / In-context adaptation (Zhang et al., 2021).

ARM uses a meta-learning objective (Hochreiter et al., 2001; Finn et al., 2017):

$$\min_{w, \phi} \frac{1}{B} \sum_{b=1}^B \frac{1}{M} \sum_{(x_m, y_m) \in \mathcal{D}_M^b} \mathcal{L}_{CE}(y_m, g(x_m; h_\phi(w, x_{1:M}^b))), \quad (5)$$

where $\mathcal{D}_M^b$ is a random subset of $M < N$ pairs of batch b , considered as *context set*. M is usually in the size of the expected future dataset from the new domain, in our case the size of an experimental batch.

The central part is the test-time adaptation function

$$h_\phi : \mathcal{W} \times \mathcal{X}^M \rightarrow \mathcal{W}, \quad w^\beta = h_\phi(w, x_{1:M}^\beta), \quad (6)$$

which maps the current model parameters w and *context* (the target-batch samples $x_{1:M}^\beta$) to the adapted parameters w^β . The test-time adaptation function h_ϕ has itself parameters ϕ , which are learned with the usual gradient descent techniques. At test time on a new experimental batch β , we first compute $w^\beta = h_\phi(w, x_{1:M}^\beta)$ and then predict with $\hat{y} = g(x; w^\beta)$ for any x from batch β .

ARM-BN. In this variant of ARM, the adaptation function h_ϕ only replaces the batch normalization statistics μ_{BN}, σ_{BN} of all batchnorm-layers of the main network g with the statistics from the batch β from the new domain.

ARM-CML. In this version, the adaptation function h_ϕ uses the unlabeled samples from the new batch to compute a compact representation, a *context vector*, that captures the specific characteristics of the target domain. Unlike ARM-BN, which only updates normalization statistics, ARM-CML performs a lightweight parameter adaptation conditioned on the new data, enabling the model to better align with the target domain without requiring labels or retraining.

3. CS-ARM-BN: Control-Stabilized Adaptive Risk Minimization with Batch Normalization

A key limitation of BN-based test-time and meta-learned adaptation methods is their reliance on statistics estimated from a set of *unlabeled target samples*. When the number of available samples is small or when the class distribution at test time differs from that observed during training, the resulting batch normalization (BN) statistics become noisy and confounded by label shift (Park et al., 2023; Zhao et al., 2023), leading to unstable or degraded performance. We confirm this occurrence in our experiments shown in Sections 5.3 and 5.4 and Appendix Sections E.1 and E.2.

In high-content screening experiments, and generally in biomedical experiments, *negative control samples* are available (Chandrasekaran et al., 2023; Arevalo et al., 2024). For every experimental batch β , a set of control images $\{z_c^\beta\}_{c=1}^C$ is acquired under the same technical conditions as the perturbed samples, but without inducing any biological effect. These controls provide a direct estimate of batch-specific technical variation that is independent of the class composition of the perturbed samples.

Control-stabilized adaptation. In CS-ARM-BN, we extend ARM-BN by incorporating negative control samples into the context used for adaptation. Instead of estimating BatchNorm (BN) statistics solely from the unlabeled perturbed samples $x_{1:L}^\beta$, we compute the adaptation statistics using the union of control and perturbed samples from the target batch,

$$\mathcal{C}^\beta = \{z_c^\beta\}_{c=1}^C \cup \{x_l^\beta\}_{l=1}^L.$$

Concretely, for each BN layer, the adapted mean and variance are computed over this combined context set as

$$\mu_{BN}^\beta = \frac{1}{|\mathcal{C}^\beta|} \sum_{u \in \mathcal{C}^\beta} u, \quad (7)$$

$$(\sigma_{BN}^\beta)^2 = \frac{1}{|\mathcal{C}^\beta|} \sum_{u \in \mathcal{C}^\beta} (u - \mu_{BN}^\beta)^2, \quad (8)$$

where $u \in \mathcal{C}^\beta$ denotes the BN-layer activations corresponding to either a control image z_c^β or a perturbed image x_m^β . These adapted statistics replace the running training statistics in all BN layers of the prediction model g when evaluating samples from batch β .

Another option to mitigate the effects of label shift and small batch sizes could be to use only control samples to adapt the Batch Normalization statistics, as proposed by Batch Effect Normalization (BEN) (Lin and Lu, 2022). However, in the following paragraph, we argue that including both perturbed and control samples provides the best trade-off between

bias and variance. Empirically, we found that adapting BN layers using control samples only at evaluation time performed poorly, but when only controls were already used during training, the performance improved (see Table 4). We evaluate BEN and its meta-learned variant ARM-BEN in our most challenging setting (Section 5.4), which combines severe label shift with cross-source transfer. This is the regime in which the bias-variance trade-off between the three estimators (see below) distinguishes them the most. Under mild label shift, the bias that BEN seeks to avoid by accepting a higher variance is small to begin with, leaving BEN dominated by ARM-BN, which is an outcome the bias-variance analysis already predicts.

Let $M = C + L$ be the total number of samples in a new target batch, where C is the number of control samples and L is the number of non-control or perturbed samples. We model the BN activation mean for sample x as:

$$\mu_{\text{obs},x} = \mu_{\text{domain}} + \mu_{\text{class}(x)} + \varepsilon_x$$

where μ_{domain} is the batch-specific technical offset we wish to estimate, $\mu_{\text{class}(x)} = 0$ for controls, $\mu_{\text{class}(x)} \neq 0$ for perturbed samples, and $\varepsilon_x \sim (0, \sigma^2)$ is additive noise.

The three estimators of μ_{domain} and their mean squared errors (MSEs) are:

1. **ARM-BN or AdaBN estimator (perturbed samples only):**

$$\text{MSE}(\hat{\mu}^{\text{ARM-BN}}) = \underbrace{\bar{\mu}_{\text{class}}^2}_{\text{large bias}^2} + \frac{\sigma^2}{L} \quad (9)$$

The bias $\bar{\mu}_{\text{class}}$ grows with the degree of label shift, explaining the performance collapse observed in Tables 3 and 4.

2. **Control-based estimator (controls only):**

$$\text{MSE}(\hat{\mu}^{\text{ctrl}}) = \underbrace{0}_{\text{no bias}} + \frac{\sigma^2}{C} \quad (10)$$

The estimator is unbiased, but suffers from high variance when C , the number of controls, is small.

3. **Our estimator CS-ARM-BN (controls + perturbed samples):**

$$\text{MSE}(\hat{\mu}^{\text{CS-ARM-BN}}) = \underbrace{\left(\frac{L}{M}\right)^2 \bar{\mu}_{\text{class}}^2}_{\text{small bias}^2} + \frac{\sigma^2}{M} \quad (11)$$

exploits all available samples $M = C + L$ and allows for a small bias, but decreases variance by a factor of $1/M$.

In typical settings, CS-ARM-BN achieves the lowest MSE by trading a small residual bias, shrunk by the factor $(L/M)^2$ relative to ARM-BN, against a reduced variance, since $M > C$. This bias-variance tradeoff is most favorable precisely in the operating regime of real drug screening: availability of few controls (C) and moderate-to-severe label shift, which is confirmed empirically in Table 4. Details in Appendix G.

Relation to ARM. Importantly, our CS-ARM-BN method remains fully within the Adaptive Risk Minimization (ARM) framework. As shown in Figure 2, during training, the model is meta-learned episodically: for each source batch b , control samples and perturbed samples are used as context to adapt BN statistics, and performance is optimized on perturbed query samples from the same batch. This trains the base network to *expect* and *exploit* control-conditioned normalization at test time, rather than treating it as a heuristic post-hoc correction.

Compared to standard ARM-BN and other domain adaptation methods, CS-ARM-BN offers three key advantages:

- **Stability under small context sizes:** since all samples, perturbed plus control samples are used in each batch, the BN statistics can be estimated reliably even when only few perturbed samples are available.
- **Robustness to label shift:** control samples are independent of the class distribution in the new domain, and are used to decrease the bias in the estimator at test time.
- **No additional supervision or re-training:** as in ARM-BN, adaptation remains fully unlabeled and lightweight, requiring only a single forward pass over the samples from the new domain i.e. experimental batch, and no expensive test-time training.

4. Related work

Traditional batch effect correction methods. Batch effect correction methods that are based on either linear transformations (Johnson et al., 2007; Ando et al., 2017), nearest neighbors (Haghverdi et al., 2018; Stuart et al., 2019; Hie et al., 2019) or a mixture of PCA and clustering (Korsunsky et al., 2019) have been previously proposed. One of the closest related works is Arevalo et al. (2024), who investigate batch correction methods for handcrafted features for microscopy images. These methods, however, have been originally designed to correct batch effects in feature vectors and cannot be used together with a model trained end-to-end directly from images.

Deep learning and batch effects in microscopy imaging data. While Lin and Lu (2022) also proposed using negative

Table 1. MoA prediction accuracy on **in-domain** and **unseen experimental batches** from a **new domain**. Column "TTA" indicates that the method can adapt itself given samples from a new domain at test time, i.e. test time adaptation. "In-context" means that the model adapts its parameters based on the test samples through in-context learning. The accuracy values denote mean and standard deviation over 5-fold cross-validation, thus five training re-runs of each method. "FM" indicates a foundation model.

Method category	Method	In-domain accuracy	New experimental batch accuracy	TTA	In-context
Baselines	no adaptation (BatchNorm)	0.939 ± 0.005	0.862 ± 0.060	✗	✗
	no adaptation (InstanceNorm)	0.949 ± 0.007	0.891 ± 0.043	✗	✗
	CA-MAE (FM) (Kraus et al., 2024)	0.747 ± 0.006	0.709 ± 0.041	✗	✗
	CellProfiler (Stirling et al., 2021)	0.923 ± 0.031	0.860 ± 0.032	✗	✗
Adversarial	DANN (Ganin and Lempitsky, 2014)	-	0.911 ± 0.020	✗	✗
Distance	CORAL (Sun et al., 2015)	-	0.893 ± 0.054	✗	✗
	CA-MAE (FM) + TVN (Ando et al., 2017)	-	0.747 ± 0.013	✓	✗
	TENT (Wang et al., 2021)	-	0.929 ± 0.013^a	✓	✗
	DEYO (Lee et al., 2024)	-	0.929 ± 0.013^a	✓	✗
	ROID (Marsden et al., 2023)	-	0.929 ± 0.013^a	✓	✗
	AdaBN (Li et al., 2016)	-	0.928 ± 0.013	✓	✗
Generative	Style-ID (Chung et al., 2024)	-	0.781 ± 0.015	✓	✗
Meta-learning	ARM-CML (Zhang et al., 2021)	-	0.843 ± 0.074	✓	✓
	ARM-BN (Zhang et al., 2021)	-	0.935 ± 0.018	✓	✓
	CS-ARM-BN (ours)	-	0.930 ± 0.019	✓	✓

^a In this mild-shift, label-balanced regime, the selection and weighting mechanisms of DEYO and ROID practically reduce to TENT's plain entropy update on the BatchNorm affine parameters; accuracies differ only at the fourth decimal.

controls to estimate BN statistics, their work is not framed as meta-learning, and they do not show the scenarios where control-based statistics are beneficial in comparison to using only perturbed samples. Palma et al. (2025); Lopez et al. (2018); Chung et al. (2024) presented generative models that could be used to address batch effects. Due to computational constraints, we use Chung et al. (2024) as representative for this class of methods.

In-context learning for biological data. Recently, in the field of single cell transcriptomic data, Dong et al. (2026) introduces a foundation model that can perform in-context predictions from unlabeled cells. However, their work focuses on tabular data and the transformer architecture.

Test-time adaptation and robustness to label shift. Aside from the test-time adaptation methods compared in this work, additional TTA methods are available. Wang et al. (2022); Gong et al. (2022); Boudiaf et al. (2022) propose methods to overcome catastrophic forgetting in online TTA, while our proposed method does not consist on continually adapting the model weights. Park et al. (2023) introduced a hypernetwork to address label shift, but it needs additional training of the model and it is not specifically tailored for biological data where control samples are available.

5. Experiments and Results

We performed a set of experiments to evaluate the effectiveness of a wide selection of methods for addressing batch

effects under different scenarios: **a) Mild domain shift: new experimental batches.** We evaluate adaptation performance when the target data originate from previously unseen batches generated under similar experimental conditions, reflecting routine batch-to-batch variability. **b) Strong domain shift: transfer across sources.** We consider a more challenging setting in which the target data come from different institutions or data-generating sources, inducing substantial domain shifts due to variations in protocols, equipment, and experimental practices. **c) Realistic scenario 1: new experimental batch with label shifts.** We study the impact of changes in class proportions, evaluating how well methods maintain performance when label distributions differ at inference time. This is the most realistic scenario, since novel sets of compounds are usually screened, which do not maintain the label distribution of the training data. One obvious example is drug discovery efforts, where the compounds that are tested in the lab are suspected to be active towards one specific protein target. **d) Realistic scenario 2: new imaging source with label shifts.** Finally, we evaluate the performance of our method and the baselines in a setting where test samples exhibit both class distribution shift and domain shift, but are additionally acquired from a new imaging source, yielding a more challenging scenario than the previous one.

Dataset. We conduct experiments on the large JUMP-CP dataset (Chandrasekaran et al., 2023). We use source 3, and source 8 in Exp. 5.2, and perform a classification task to

Table 2. Comparison of MoA classification accuracy of different methods across target sources. The element before the arrow indicates the training domain, the element after the arrow indicates the evaluation domain. The accuracy values denote the mean and standard deviation over 3 re-runs with different seeds.

Method	Source $\rightarrow$ Target			
	S3 $\rightarrow$ S3	S3 $\rightarrow$ S8	S8 $\rightarrow$ S8	S8 $\rightarrow$ S3
no adaptation (BatchNorm)	0.939 $\pm$ 0.005	0.670 $\pm$ 0.016	0.974 $\pm$ 0.002	0.498 $\pm$ 0.011
no adaptation (InstanceNorm)	0.949 $\pm$ 0.007	0.796 $\pm$ 0.018	0.984 $\pm$ 0.001	0.552 $\pm$ 0.014
CA-MAE (FM)	0.747 $\pm$ 0.006	0.389 $\pm$ 0.015	0.840 $\pm$ 0.004	0.356 $\pm$ 0.011
CellProfiler	0.923 $\pm$ 0.031	0.430 $\pm$ 0.074	0.991 $\pm$ 0.004	0.550 $\pm$ 0.029
DANN	–	0.639 $\pm$ 0.175	–	0.324 $\pm$ 0.079
CORAL	–	0.804 $\pm$ 0.018	–	0.697 $\pm$ 0.010
CA-MAE (FM) + TVN	–	0.688 $\pm$ 0.005	–	0.488 $\pm$ 0.010
TENT	–	0.835 $\pm$ 0.033	–	0.785 $\pm$ 0.008
DEYO	–	0.836 $\pm$ 0.033	–	0.784 $\pm$ 0.007
ROID	–	0.835 $\pm$ 0.034	–	0.788 $\pm$ 0.010
AdaBN	–	0.838 $\pm$ 0.037	–	0.788 $\pm$ 0.003
StyleID	–	0.774 $\pm$ 0.004	–	0.713 $\pm$ 0.016
ARM-CML	–	0.645 $\pm$ 0.034	–	0.569 $\pm$ 0.030
ARM-BN	–	0.874 $\pm$ 0.008	–	0.795$\pm$0.002
CS-ARM-BN (ours)	–	0.884$\pm$0.005	–	0.776 $\pm$ 0.016

predict eight well-defined MoAs. Therefore, we work with samples perturbed with eight different compounds, each one of them known to produce one of these MoAs (see Appendix Table A6), and which are present in every plate of the dataset. Each microscopy image has five channels; we use the preprocessing described in Appendix Section C.

Data splits. To evaluate robustness to batch effects, we consider two data splitting strategies reflecting different levels of domain shift (see Appendix Section E.4). In most experiments, we adopt an *experimental-batch split*, where training and evaluation are performed on disjoint experimental batches, representing a challenging but realistic batch generalization setting. For the strong domain shift scenario, we instead evaluate on images acquired from a completely unseen imaging *source*, corresponding to a cross-platform or cross-institution transfer setting. Under the experimental-batch split, we use **5-fold cross-validation** and report **mean accuracy $\pm$ standard deviation** across folds, corresponding to five independent training runs per method. Unless otherwise specified, accuracy is reported at the image level.

Methods and implementation. We compare supervised baselines (ResNet50 with BN/IN; CA-MAE linear probe; CellProfiler features), classical UDA/MSDA baselines (DANN, CORAL), generative style transfer (Style-ID), and test-time adaptation / in-context methods (AdaBN, TENT, DEYO, ROID, ARM-CML, ARM-BN). Training details and hyperparameter search spaces are provided in Appendix Table A5. For methods that require *test-time* samples to

adapt (AdaBN, TENT, ARM-BN, ARM-CML), we evaluate them in a strictly **unlabeled** test-time adaptation setting unless explicitly stated.

5.1. Mild domain shift: new experimental batches within a source

We first study the most common practical scenario: training and test data originate from the *same* imaging source, but the test batches correspond to *new experimental batches*. This setting reflects *mild* domain shift: acquisition conditions drift across experiments, but the imaging platform remains unchanged. Results are summarized in Table 1.

Batch effects degrade generalization performance. A standard ResNet50 achieves high in-domain accuracy (e.g., 0.939 ± 0.005 with BN), but drops substantially when evaluated on unseen experimental batches (0.862 ± 0.060). This gap quantifies the severity of batch effects even under mild shifts.

Classical UDA baselines are insufficient or impractical. Domain-invariant learning (DANN) and discrepancy/moment matching (CORAL) do not reliably close the gap in this few-shot multi-source regime. Moreover, these approaches conceptually assume access to the target distribution during training and are thus inefficient in sequential settings where new batches continually arrive.

Test-time and in-context adaptation close the gap. Methods that adapt using the unlabeled target batch at inference

time provide the strongest gains. In particular, adapting normalization statistics (AdaBN) and entropy-minimizing BN adaptation (TENT) markedly improve robustness. Meta-learned in-context approaches further improve performance: ARM-BN achieves the best accuracy (Table 1), approaching the in-domain baseline and thereby largely neutralizing mild batch effects. This supports our hypothesis that, in microscopy, batch effects can often be corrected by *fast, context-conditioned* adaptation rather than retraining.

5.2. Strong domain shift: transfer across sources

Next, we evaluate whether the above conclusions hold under *strong* domain shift across laboratories, which might have different data acquisition protocols and microscopes. We consider the setting where the main model is trained using labeled images obtained from only one source, i.e. a pharmaceutical company, and the evaluation is performed on samples from another source. Specifically, we use source 3 (S3) and source 8 (S8), and we perform experiments where the source (labeled) domain is S3 and the target domain is S8, and vice versa, and we also compare to the in-domain performance for each of them. Table 2 reports MoA accuracy for each method under source transfer.

Cross-source shifts are substantially harder. All methods degrade under cross-source transfer compared to within-source evaluation, confirming that laboratory-to-laboratory changes introduce stronger shifts than batch-to-batch drift within a single source.

Normalization-based adaptation only at test time remains competitive but is less stable. AdaBN and TENT improve over non-adaptive baselines in some transfer directions, but the improvements are not consistent across all source→target pairs. This variability is expected: under strong shift, the target feature statistics may differ not only due to style or illumination differences, but also due to more complex, structured differences (e.g., staining intensity distributions, microscope optics, and site-specific pipelines), and may require more robust context construction.

Meta-learned in-context adaptation improves cross-source robustness. ARM-BN tends to provide stronger cross-source performance than purely test-time methods because the model is trained to adapt to new domains via episodic context sampling during training. Overall, Experiment 5.2 highlights that cross-source robustness remains challenging, but *learning to adapt* is a promising direction compared to static training or target-dependent retraining.

5.3. Realistic scenario 1: new experimental batch with label shifts

A known failure mode of BN-based adaptation is *label shift* (i.e., changes in the class proportions between training and

Table 3. Comparison of MoA classification accuracy of different methods in a new experimental batch under different degrees of label shift. A smaller α value corresponds to a larger degree of label shift.

Method	Label shift degree		
	$\alpha = 1$	$\alpha = 0.1$	$\alpha = 0.01$
ERM (no adaptation)	0.862 ± 0.060	0.862 ± 0.061	0.862 ± 0.061
TENT	0.832 ± 0.014	0.778 ± 0.016	0.156 ± 0.010
DEYO	0.830 ± 0.014	0.773 ± 0.015	0.156 ± 0.010
ROID	0.837 ± 0.017	0.792 ± 0.020	0.157 ± 0.011
AdaBN	0.851 ± 0.016	0.452 ± 0.010	0.229 ± 0.009
ARM-BN	0.831 ± 0.017	0.416 ± 0.020	0.228 ± 0.009
CS-ARM-BN (ours)	0.924 ± 0.018	0.906 ± 0.021	0.894 ± 0.022

test) (Park et al., 2023; Zhao et al., 2023). In microscopy screening, this occurs naturally: many experiments are designed to test only a narrow set of hypotheses, and thus only a subset of MoAs may appear in a new batch. If the adaptation statistics are computed over the *perturbed* samples, then changing the class mixture changes the feature statistics and can mislead adaptation.

Protocol. We simulate increasing levels of label shift by subsampling the target batch to enforce different class-mixture skew strengths. We report performance at three shift degrees (Table 3): from mild to severe label shift. In all of the experiments, we use a constant number of labeled samples so the decrease in accuracy is not due to a smaller batch size. In every TTA method, the adaptation is unlabeled; only the class mixture is manipulated. While for TENT, AdaBN and ARM-BN, the accuracy decreases rapidly when label shift increases, the performance for CS-ARM-BN remains robust. Moreover, in Supplementary Table A9 we also compare to AdaBN and ARM-BN including negative controls so that every method uses the same amount of samples to compute the BN statistics, and it can be seen that CS-ARM-BN still presents the highest performance.

BN-adaptation degrades under label shift. As label shift increases, AdaBN, TENT and ARM-BN lose accuracy because their estimated BN statistics entangle (i) batch-specific nuisance variation with (ii) class-mixture-specific feature shifts. In other words, the adaptation signal becomes confounded by the changing MoA composition.

Controls stabilize adaptation under label shift. CS-ARM-BN addresses this confounding by computing the adaptation statistics from both perturbed samples and *negative controls*, which are independent of the MoA mixture by design. Consequently, the adaptation remains anchored to a consistent reference distribution per batch, and performance becomes substantially more stable across label-shift degrees. In Appendix Section E.1, we show that CS-ARM-BN stabilizes the performance under small batch sizes.

Table 4. Comparison of MoA classification accuracy of different methods across target sources (S8, S3, and S6) under different degrees of label shift. A lower α denotes a higher degree of label shift. The accuracy values denote mean and standard deviation over re-runs with three different seeds.

Method	S3 $\rightarrow$ S8		S8 $\rightarrow$ S3		S3 $\rightarrow$ S6		S8 $\rightarrow$ S6	
	$\alpha = 1$	$\alpha = 0.01$	$\alpha = 1$	$\alpha = 0.01$	$\alpha = 1$	$\alpha = 0.01$	$\alpha = 1$	$\alpha = 0.01$
no adaptation (BN)	0.669 $\pm$ 0.016	0.670 $\pm$ 0.016	0.500 $\pm$ 0.012	0.498 $\pm$ 0.012	0.537 $\pm$ 0.069	0.536 $\pm$ 0.069	0.568 $\pm$ 0.005	0.568 $\pm$ 0.006
TENT	0.706 $\pm$ 0.055	0.139 $\pm$ 0.006	0.659 $\pm$ 0.005	0.136 $\pm$ 0.000	0.552 $\pm$ 0.027	0.135 $\pm$ 0.004	0.571 $\pm$ 0.006	0.122 $\pm$ 0.001
DEYO	0.705 $\pm$ 0.056	0.139 $\pm$ 0.006	0.658 $\pm$ 0.007	0.137 $\pm$ 0.000	0.552 $\pm$ 0.027	0.135 $\pm$ 0.004	0.572 $\pm$ 0.004	0.122 $\pm$ 0.001
ROID	0.715 $\pm$ 0.050	0.138 $\pm$ 0.004	0.663 $\pm$ 0.005	0.135 $\pm$ 0.000	0.557 $\pm$ 0.025	0.135 $\pm$ 0.004	0.574 $\pm$ 0.006	0.123 $\pm$ 0.001
AdaBN	0.736 $\pm$ 0.046	0.171 $\pm$ 0.009	0.688 $\pm$ 0.004	0.164 $\pm$ 0.002	0.580 $\pm$ 0.023	0.159 $\pm$ 0.006	0.586 $\pm$ 0.005	0.149 $\pm$ 0.002
BEN (w/o meta-learning)	0.125 $\pm$ 0.000	0.125 $\pm$ 0.000	0.125 $\pm$ 0.000	0.125 $\pm$ 0.000	0.124 $\pm$ 0.001	0.122 $\pm$ 0.004	0.125 $\pm$ 0.000	0.125 $\pm$ 0.000
ARM-BEN	0.664 $\pm$ 0.031	0.663 $\pm$ 0.031	0.544 $\pm$ 0.031	0.543 $\pm$ 0.031	0.547 $\pm$ 0.040	0.546 $\pm$ 0.040	0.566 $\pm$ 0.024	0.565 $\pm$ 0.024
ARM-BN	0.747 $\pm$ 0.010	0.173 $\pm$ 0.004	0.680 $\pm$ 0.007	0.159 $\pm$ 0.001	0.550 $\pm$ 0.011	0.158 $\pm$ 0.003	0.579 $\pm$ 0.010	0.154 $\pm$ 0.003
CS-ARM-BN (ours)	0.878$\pm$0.006	0.825$\pm$0.007	0.788$\pm$0.014	0.730$\pm$0.015	0.657$\pm$0.003	0.606$\pm$0.006	0.648$\pm$0.011	0.611$\pm$0.015

5.4. Realistic scenario 2: new imaging source with label shifts

As in the previous experiment, the evaluation samples in this scenario exhibit a class distribution that differs from the training data. However, in this case, the domain shift is more pronounced, as the test samples originate from a new imaging source. In Table 4, we show that, similarly to Experiment 5.3, the performance of methods that do not leverage negative controls (TENT, AdaBN, and ARM-BN) degrades as the degree of label shift increases.

We evaluate the performance of BEN, which relies exclusively on control samples for adaptation, for which we also extend BEN to meta-learning. While the version without meta-learning performs poorly, the meta-learned variant, despite achieving lower accuracy than CS-ARM-BN, outperforms the other methods under strong label shift.

Consistent with the previous experiment, CS-ARM-BN stabilizes the batch normalization statistics and achieves the highest accuracy across both mild and severe domain shifts.

6. Discussion and Limitations

We framed batch effect correction for mechanism of action (MoA) classification as a *multi-source domain adaptation* problem, reflecting a real use-case scenario of high throughput microscopy imaging data. Our study shows that *in-context adaptation*, where models condition their parameters on a small set of unlabeled samples from a new batch, outperforms established approaches.

Limitations. Although we have shown that CS-ARM-BN presents a robust performance, especially under label shifts and when the number of perturbed samples is small, our study is limited to the experimental settings considered here and does not cover other scenarios. For example, we did not consider the setting where the controls come from a different cell line than the perturbed samples. Another note-

worthy experimental setting is that we process each image from a certain well individually. Processing all well images together may make the results more stable, alleviating the impact of noisy views. Moreover, additional strategies like test-time scaling might further improve performance, as shown in Supplementary Section I.

Experimental findings. Across four experiments, we find that (i) batch effects cause a large generalization gap even under mild within-source shifts, (ii) cross-source transfer is substantially harder but benefits from meta-learned in-context adaptation, and (iii) standard BN-based test-time adaptation methods are weak when target statistics are estimated from few or label-skewed perturbed samples. By leveraging the experimental design of high-content screening, i.e. the presence of negative controls in every batch, CS-ARM-BN yields strong accuracy while remaining robust to small context sizes and label shift. We intentionally focus on lightweight, fast adaptation methods that can operate with limited target data and minimal computation, reflecting realistic deployment conditions.

In the evaluated settings, ARM-BN and CS-ARM-BN nearly close the performance gap between in-domain and out-of-domain evaluation. On unseen experimental batches, ARM-BN and CS-ARM-BN reach 0.935 ± 0.018 and 0.930 ± 0.019 accuracy, respectively, matching the 0.939 ± 0.005 accuracy observed within the training distribution. This demonstrates that batch effects, long a bottleneck for robust MoA prediction, can be effectively neutralized. Our findings highlight that in-context test-time domain adaptation methods can make deep learning models for MoA identification not only accurate but also practical for deployment across experimental batches.

Software and data

Code and data are available at <https://github.com/ml-jku/cs-arm-bn>.

Acknowledgments

The ELLIS Unit Linz, the LIT AI Lab, the Institute for Machine Learning, are supported by the Federal State Upper Austria. We thank the projects FWF AIRI FG 9-N (10.55776/FG9), AI4GreenHeatingGrids (FFG- 899943), Stars4Waters (HORIZON-CL6-2021-CLIMATE-01-01), FWF Bilateral Artificial Intelligence (10.55776/COE12). We thank Merck Healthcare KGaA, NXAI GmbH, Audi AG, TÜV Holding GmbH, Software Competence Center Hagenberg GmbH, dSPACE GmbH, TRUMPF SE + Co. KG. TP is funded by the European Union, EIC-2023-PATHFINDEROPEN-01 (I-SCREEN, grant no. 101130093). Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union or European Innovation Council and SMEs Executive Agency (EISMEA). Neither the European Union nor the granting authority can be held responsible for them.

Impact Statement

This work addresses one of the central barriers to deploying machine learning in real-world biomedical settings: poor generalization across experimental batches and domains. We show that widely used domain adaptation strategies can fail under realistic conditions involving batch effects and label shift, and we introduce CS-ARM-BN, a method that explicitly leverages negative control samples to estimate and correct domain-specific shifts in a principled and stable way.

The impact of this work is both methodological and translational. Methodologically, it highlights the limitations of assuming covariate shift alone in scientific imaging data and demonstrates how experimental design elements (negative controls) can be incorporated directly into training and inference pipelines to address these limitations. Translationally, improved cross-batch generalization enables more reliable mechanism-of-action classification. Also, by reducing the need for extensive per-batch retraining and manual correction, our approach could lower the cost of model deployment in comparison to other domain adaptation methods.

However, in case of failure of any machine learning model employed for drug discovery, false positives can lead to costly downstream validation of ineffective compounds, while false negatives risk discarding potentially valuable therapeutic candidates early in the pipeline. Moreover, our evaluation focuses on a limited set of compounds with well-characterized and relatively strong mechanisms of action, where phenotypic signals are pronounced. Compounds with subtler effects might be more difficult to classify.

Despite these limitations, this work contributes to the development of machine learning systems that are robust to

non-i.i.d. data, which is a requirement for trustworthy AI in biology, medicine, and other experimental sciences.

References

- Ando, D. M., McLean, C. Y., and Berndl, M. (2017). Improving phenotypic measurements in high-content imaging screens. *bioRxiv*, page 161422.
- Arevalo, J., Su, E., Ewald, J. D., van Dijk, R., Carpenter, A. E., and Singh, S. (2024). Evaluating batch correction methods for image-based cell profiling. *Nature Communications* 2024 15:1, 15:1–12.
- Baxter, J. (1998). Theoretical models of learning to learn. In *Learning to learn*, pages 71–94. Springer.
- Ben-David, S., Blitzer, J., Crammer, K., and Pereira, F. (2006). Analysis of representations for domain adaptation. *Advances in neural information processing systems*, 19.
- Blanchard, G., Lee, G., and Scott, C. (2011). Generalizing from several related classification tasks to a new unlabeled sample. In Shawe-Taylor, J., Zemel, R., Bartlett, P., Pereira, F., and Weinberger, K., editors, *Advances in Neural Information Processing Systems*, volume 24. Curran Associates, Inc.
- Boudiaf, M., Mueller, R., Ayed, I. B., and Bertinetto, L. (2022). Parameter-free online test-time adaptation. *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition*, 2022-June:8334–8343.
- Bousmalis, K., Silberman, N., Dohan, D., Erhan, D., and Krishnan, D. (2017). Unsupervised pixel-level domain adaptation with generative adversarial networks. *Proceedings - 30th IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2017*, 2017-January:95–104.
- Bray, M.-A., Carpenter, A., of MIT, B. I., and Platform, H. I. (2017). Advanced assay development guidelines for image-based high content screening and analysis. *Assay Guidance Manual*.
- Carpenter-Singh and Cimini Labs at the Broad Institute (2026). *CellPainting Gallery*. <https://registry.opendata.aws/cellpainting-gallery/>. [Online; accessed 26-May-2026].
- Chandrasekaran, S. N., Ackerman, J., Alix, E., Ando, D. M., Arevalo, J., Bennion, M., Boisseau, N., Borowa, A., Boyd, J. D., Brino, L., Byrne, P. J., Ceulemans, H., Ch’ng, C., Cimini, B. A., Clevert, D.-A., Deflaux, N., Doench, J. G., Dorval, T., Doyonnas, R., Dragone, V., Engkvist, O., Faloon, P. W., Fritchman, B., Fuchs, F., Garg, S., Gilbert, T. J., Glazer, D., Gnut, D., Goodale, A., Grignard, J.,

- Guenther, J., Han, Y., Hanifehlou, Z., Hariharan, S., Hernandez, D., Horman, S. R., Hormel, G., Huntley, M., Icke, I., Iida, M., Jacob, C. B., Jaensch, S., Khetan, J., Kost-Altimova, M., Krawiec, T., Kuhn, D., Lardeau, C.-H., Lembke, A., Lin, F., Little, K. D., Lofstrom, K. R., Lotfi, S., Logan, D. J., Luo, Y., Madoux, F., Zapata, P. A. M., Marion, B. A., Martin, G., McCarthy, N. J., Mervin, L., Miller, L., Mohamed, H., Monteverde, T., Mouchet, E., Nicke, B., Ogier, A., Ong, A.-L., Osterland, M., Otrocka, M., Peeters, P. J., Pilling, J., Prechtl, S., Qian, C., Rataj, K., Root, D. E., Sakata, S. K., Scrace, S., Shimizu, H., Simon, D., Sommer, P., Spruiell, C., Sumia, I., Swalley, S. E., Terauchi, H., Thibaudeau, A., Unruh, A., de Waeter, J. V., Dyck, M. V., van Staden, C., Warchoř, M., Weisbart, E., Weiss, A., Wiest-Daessle, N., Williams, G., Yu, S., Zapiec, B., Źyła, M., Singh, S., and Carpenter, A. E. (2023). Jump cell painting dataset: morphological impact of 136,000 chemical and genetic perturbations. *bioRxiv*, page 2023.03.23.534023.
- Chen, W., Zhao, Y., Chen, X., Yang, Z., Xu, X., Bi, Y., Chen, V., Li, J., Choi, H., Ernest, B., Tran, B., Mehta, M., Kumar, P., Farmer, A., Mir, A., Mehra, U. A., Li, J. L., Moos, M., Xiao, W., and Wang, C. (2020). A multicenter study benchmarking single-cell rna sequencing technologies using reference samples. *Nature Biotechnology* 2020 39:9, 39:1103–1114.
- Chung, J., Hyun, S., and Heo, J. P. (2024). Style injection in diffusion: A training-free approach for adapting large-scale diffusion models for style transfer. *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition*, pages 8795–8805.
- Dong, M., Adduri, A., Gautam, D., Carpenter, C., Shah, R., Ricci-Tam, C., Kluger, Y., Burke, D. P., and Roohani, Y. H. (2026). Stack: In-context learning of single-cell biology. *bioRxiv*.
- Farahani, A., Voghoei, S., Rasheed, K., and Arabnia, H. R. (2020). A brief review of domain adaptation. *ArXiv*, pages 877–894.
- Finn, C., Abbeel, P., and Levine, S. (2017). Model-agnostic meta-learning for fast adaptation of deep networks. In *International conference on machine learning*, pages 1126–1135. PMLR.
- Ganin, Y. and Lempitsky, V. (2014). Unsupervised domain adaptation by backpropagation. *32nd International Conference on Machine Learning, ICML 2015*, 2:1180–1189.
- Gong, T., Jeong, J., Kim, T., Kim, Y., Shin, J., and Lee, S. J. (2022). Note: Robust continual test-time adaptation against temporal correlation. *Advances in Neural Information Processing Systems*, 35.
- Graham, B., El-Nouby, A., Touvron, H., Stock, P., Joulin, A., Jégou, H., and Douze, M. (2021). Levit: a vision transformer in convnet’s clothing for faster inference. *Proceedings of the IEEE International Conference on Computer Vision*, pages 12239–12249.
- Guo, J., Shah, D. J., and Barzilay, R. (2018). Multi-source domain adaptation with mixture of experts. *arXiv preprint arXiv:1809.02256*.
- Haghverdi, L., Lun, A. T., Morgan, M. D., and Marioni, J. C. (2018). Batch effects in single-cell rna-sequencing data are corrected by matching mutual nearest neighbors. *Nature Biotechnology* 2018 36:5, 36:421–427.
- Haslum, J. F., Matsoukas, C., Leuchowius, K. J., and Smith, K. (2023). Bridging generalization gaps in high content imaging through online self-supervised domain adaptation. *IEEE Workshop/Winter Conference on Applications of Computer Vision*, pages 7723–7732.
- He, K., Zhang, X., Ren, S., and Sun, J. (2015). Deep residual learning for image recognition. *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition*, 2016-December:770–778.
- Hie, B., Bryson, B., and Berger, B. (2019). Efficient integration of heterogeneous single-cell transcriptomes using scanorama. *Nature biotechnology*, 37:685–691.
- Hochreiter, S., Younger, A. S., and Conwell, P. R. (2001). Learning to learn using gradient descent. In *International conference on artificial neural networks*, pages 87–94. Springer.
- Hughes, J. P., Rees, S. S., Kalindjian, S. B., and Philpott, K. L. (2011). Principles of early drug discovery. *British Journal of Pharmacology*, 162:1239.
- Ioffe, S. and Szegedy, C. (2015). Batch normalization: accelerating deep network training by reducing internal covariate shift. In *Proceedings of the 32nd International Conference on International Conference on Machine Learning - Volume 37, ICML’15*, page 448–456. JMLR.org.
- Johnson, W. E., Li, C., and Rabinovic, A. (2007). Adjusting batch effects in microarray expression data using empirical bayes methods. *Biostatistics (Oxford, England)*, 8:118–127.
- Kim, V., Adaloglou, N., Osterland, M., Morelli, F. M., Halawa, M., König, T., Gnutt, D., and Zapata, P. A. M. (2025). Self-supervision advances morphological profiling by unlocking powerful image representations. *Scientific Reports* 2025 15:1, 15:1–15.
- Kimura, M. and Hino, H. (2024). A short survey on importance weighting for machine learning. *arXiv preprint arXiv:2403.10175*.

- Knowles, J. and Gromo, G. (2003). A guide to drug discovery: Target selection in drug discovery. *Nature reviews. Drug discovery*, 2:63–69.
- Korsunsky, I., Millard, N., Fan, J., Slowikowski, K., Zhang, F., Wei, K., Baglaenko, Y., Brenner, M., Loh, P., and Raychaudhuri, S. (2019). Fast, sensitive and accurate integration of single-cell data with harmony. *Nature Methods* 2019 16:12, 16:1289–1296.
- Kraus, O., Kenyon-Dean, K., Saberian, S., Fallah, M., McLean, P., Leung, J., Sharma, V., Khan, A., Balakrishnan, J., Celik, S., Beaini, D., Sypetkowski, M., Cheng, C. V., Morse, K., Makes, M., Mabey, B., and Earnshaw, B. (2024). Masked autoencoders for microscopy are scalable learners of cellular biology. *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition*, pages 11757–11768.
- Lee, J., Jung, D., Lee, S., Park, J., Shin, J., Hwang, U., and Yoon, S. (2024). Entropy is not enough for test-time adaptation: From the perspective of disentangled factors. *12th International Conference on Learning Representations, ICLR 2024*.
- Leek, J. T., Scharpf, R. B., Bravo, H. C., Simcha, D., Langmead, B., Johnson, W. E., Geman, D., Baggerly, K., and Irizarry, R. A. (2010). Tackling the widespread and critical impact of batch effects in high-throughput data. *Nature reviews. Genetics*, 11:10.1038/nrg2825.
- Li, Y., Wang, N., Shi, J., Liu, J., and Hou, X. (2016). Revisiting batch normalization for practical domain adaptation. *International Conference on Learning Representations*.
- Lin, A. and Lu, A. (2022). Incorporating knowledge of plates in batch normalization improves generalization of deep learning for microscopy images. In Knowles, D. A., Mostafavi, S., and Lee, S.-I., editors, *Proceedings of the 17th Machine Learning in Computational Biology meeting*, volume 200 of *Proceedings of Machine Learning Research*, pages 74–93. PMLR.
- Liu, M.-Y. and Tuzel, O. (2016). Coupled generative adversarial networks. In Lee, D., Sugiyama, M., Luxburg, U., Guyon, I., and Garnett, R., editors, *Advances in Neural Information Processing Systems*, volume 29. Curran Associates, Inc.
- Lopez, R., Regier, J., Cole, M. B., Jordan, M. I., and Yosef, N. (2018). Deep generative modeling for single-cell transcriptomics. *Nature methods*, 15:1053–1058.
- Luecken, M. D., Büttner, M., Chaichoompu, K., Danese, A., Interlandi, M., Mueller, M. F., Strobl, D. C., Zappia, L., Dugas, M., Colomé-Tatché, M., and Theis, F. J. (2021). Benchmarking atlas-level data integration in single-cell genomics. *Nature Methods* 2021 19:1, 19:41–50.
- Marsden, R. A., Döbler, M., and Yang, B. (2023). Universal test-time adaptation through weight ensembling, diversity weighting, and prior correction. *Proceedings - 2024 IEEE Winter Conference on Applications of Computer Vision, WACV 2024*, pages 2543–2553.
- Palma, A., Theis, F. J., and Lotfollahi, M. (2025). Predicting cell morphological responses to perturbations using generative modeling. *Nature Communications*, 16:1–19.
- Park, S., Yang, S., Choo, J., and Yun, S. (2023). Label shift adapter for test-time adaptation under covariate and label shifts. *Proceedings of the IEEE International Conference on Computer Vision*, pages 16375–16385.
- Peidli, S., Green, T. D., Shen, C., Gross, T., Min, J., Garda, S., Yuan, B., Schumacher, L. J., Taylor-King, J. P., Marks, D. S., Luna, A., Blüthgen, N., and Sander, C. (2023). scPerturb: Harmonized single-cell perturbation data. *bioRxiv*, page 2022.08.20.504663.
- Peng, X., Bai, Q., Xia, X., Huang, Z., Saenko, K., and Wang, B. (2019). Moment matching for multi-source domain adaptation. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 1406–1415.
- Shimodaira, H. (2000). Improving predictive inference under covariate shift by weighting the log-likelihood function. *Journal of statistical planning and inference*, 90(2):227–244.
- Stirling, D. R., Swain-Bowden, M. J., Lucas, A. M., Carpenter, A. E., Cimini, B. A., and Goodman, A. (2021). Cellprofiler 4: improvements in speed, utility and usability. *BMC Bioinformatics*, 22:433.
- Stuart, T., Butler, A., Hoffman, P., Hafemeister, C., Papalexi, E., Mauck, W. M., Hao, Y., Stoeckius, M., Smibert, P., and Satija, R. (2019). Comprehensive integration of single-cell data. *Cell*, 177:1888–1902.e21.
- Sun, B., Feng, J., and Saenko, K. (2015). Return of frustratingly easy domain adaptation. *30th AAAI Conference on Artificial Intelligence, AAAI 2016*, pages 2058–2065.
- Sypetkowski, M., Rezanejad, M., Saberian, S., Kraus, O., Urbanik, J., Taylor, J., Mabey, B., Victors, M., Yosinski, J., Sereshkeh, A. R., Haque, I., and Earnshaw, B. (2023). RxRx1: A dataset for evaluating experimental batch correction methods. *IEEE Computer Society Conference on Computer Vision and Pattern Recognition Workshops*, 2023-June:4285–4294.
- Taigman, Y., Polyak, A., and Wolf, L. (2016). Unsupervised cross-domain image generation. *ArXiv*, abs/1611.02200.

- Venkat, N., Kundu, J. N., Singh, D., Revanur, A., et al. (2020). Your classifier can secretly suffice multi-source domain adaptation. *Advances in Neural Information Processing Systems*, 33:4647–4659.
- Wang, D., Shelhamer, E., Liu, S., Olshausen, B., and Darrell, T. (2021). TENT: Fully test-time adaptation by entropy minimization. In *International Conference on Learning Representations*.
- Wang, Q., Fink, O., Gool, L. V., and Dai, D. (2022). Continual test-time domain adaptation. *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition*, 2022-June:7191–7201.
- Wen, J., Greiner, R., and Schuurmans, D. (2020). Domain aggregation networks for multi-source domain adaptation. In *International conference on machine learning*, pages 10214–10224. PMLR.
- Yang, L., Balaji, Y., Lim, S.-N., and Shrivastava, A. (2020). Curriculum manager for source selection in multi-source domain adaptation. In *European conference on computer vision*, pages 608–624. Springer.
- Zellinger, W., Grubinger, T., Lughofer, E., Natschläger, T., and Saminger-Platz, S. (2017). Central moment discrepancy (cmd) for domain-invariant representation learning. In *International Conference on Learning Representations (ICLR)*.
- Zhang, J., Ubas, A. A., de Borja, R., Svensson, V., Thomas, N., Thakar, N., Lai, I., Winters, A., Khan, U., Jones, M. G., Tran, V., Pangallo, J., Papalexi, E., Sapre, A., Nguyen, H., Sanderson, O., Nigos, M., Kaplan, O., Schroeder, S., Hariadi, B., Marrujo, S., Salvino, C. C. A., Gallareta Olivares, G., Koehler, R., Geiss, G., Rosenberg, A., Roco, C., Merico, D., Alidoust, N., Goodarzi, H., and Yu, J. (2025). Tahoe-100M: A giga-scale single-cell perturbation atlas for context-dependent gene function and cellular modeling. *bioRxiv*.
- Zhang, M., Marklund, H., Dhawan, N., Gupta, A., Levine, S., and Finn, C. (2021). Adaptive Risk Minimization: learning to adapt to domain shift. In *Proceedings of the 35th International Conference on Neural Information Processing Systems*, NeurIPS ’21, Red Hook, NY, USA. Curran Associates Inc.
- Zhao, H., Liu, Y., Alahi, A., and Lin, T. (2023). On pitfalls of test-time adaptation. *Proceedings of Machine Learning Research*, 202:42058–42080.
- Zhao, H., Zhang, S., Wu, G., Moura, J. M., Costeira, J. P., and Gordon, G. J. (2018). Adversarial Multiple Source Domain Adaptation. *Advances in neural information processing systems*, 31.

Contents

A	Notation table	16
B	Training details	17
C	Dataset details	17
D	Additional compared methods	19
E	Additional experiments	20
E.1	Failure mode of BN adaptation — dependence on adaptation batch size	20
E.2	Label shift experiment. Supplementary information.	21
E.3	Evaluation of CS-ARM-BN with a Vision Transformer-based architecture	22
E.4	Microscopy imaging data structure and domain alignment	22
F	The in-context learning view on CS-ARM-BN	23
G	Details on estimators	23
G.1	Setup	23
G.2	Estimator 1: ARM-BN and AdaBN (Perturbed Samples Only)	24
G.3	Estimator 2: Controls Only	24
G.4	Estimator 3: CS-ARM-BN (Controls + Perturbed Samples)	24
G.5	Comparison	25
H	Detecting edge cases with uncertainty	25
I	Test-Time Scaling	26

A. Notation table

Definition	Symbol	Type
Scalars		
number of source domains (batches)	B	$\mathbb{N}$
number of samples per source domain (simplified)	N	$\mathbb{N}$
number of samples from one of the target domains	M	$\mathbb{N}$
number of control samples from target domain	C	$\mathbb{N}$
number of non-control samples from target domain	L	$\mathbb{N}$
feature dimension	D	$\mathbb{N}$
number of classes	K	$\mathbb{N}$
DANN domain-loss trade-off	λ	$\mathbb{R}_{\geq 0}$
CORAL penalty weight	γ	$\mathbb{R}_{\geq 0}$
Sets, spaces, and distributions		
input/image space	$\mathcal{X}$	
label space / class set	$\mathcal{Y}$	
training data (all sources)	$\mathcal{D}^{\text{train}}$	$\bigcup_{b=1}^B \mathcal{D}^{(b)}$
data from source domain b	$\mathcal{D}^{(b)}$	$\{(\mathbf{x}_n^{(b)}, \mathbf{y}_n^{(b)})\}_{n=1}^N$
unseen target-batch data	$\mathcal{D}^\beta$	$\{\mathbf{x}_1^\beta, \dots, \mathbf{x}_M^\beta\}$
domain-specific data distribution	p_{xy}^b	distribution on $\mathcal{X} \times \mathcal{Y}$
meta-distribution over domains	μ	distribution on domains
Representations and tensors		
image / sample / input	$\mathbf{x}$	$\mathcal{X}$
control image / control sample	$\mathbf{z}$	$\mathcal{X}$
label	$\mathbf{y}$	$\mathcal{Y}$
feature embedding	$\mathbf{a} = \mathbf{f}(\mathbf{x}; \mathbf{w}')$	$\mathbb{R}^D$
TVN-normalized embedding	$\tilde{\mathbf{a}}$	$\mathbb{R}^D$
control embeddings (batch β)	$\{\mathbf{z}_j^\beta\}$	$\mathbb{R}^D$
source/target feature covariances	$\mathbf{C}_S, \mathbf{C}_T$	$\mathbb{R}^{D \times D}$
control mean/covariance (TVN)	$\mu^{\text{ctrl}, \beta}, \Sigma^{\text{ctrl}, \beta}$	$\mathbb{R}^D, \mathbb{R}^{D \times D}$
Models and parameters		
feature extractor	$\mathbf{f}, \mathbf{f}_w, \mathbf{f}(\cdot; \mathbf{w}')$	$\mathcal{X} \rightarrow \mathbb{R}^D$
classifier	$\mathbf{c}(\cdot; \boldsymbol{\theta})$	$\mathbb{R}^D \rightarrow \mathcal{Y}$
prediction model	$\mathbf{g}(\mathbf{x}; \mathbf{w}) = (\mathbf{c} \circ \mathbf{f})(\mathbf{x}; \mathbf{w})$	$\mathcal{X} \rightarrow \mathcal{Y}$
model parameters	$\mathbf{w}$	$\mathcal{W}$
adaptation function (test-time)	$\mathbf{h}_\phi(\mathbf{w}, \mathbf{x}_{1:M}^\beta)$	$\mathcal{W} \times \mathcal{X}^M \rightarrow \mathcal{W}$
adaptation parameters	ϕ	parameters of $\mathbf{h}$
adapted parameters for batch β	$\mathbf{w}^\beta$	$\mathcal{W}$
domain discriminator (DANN)	$\mathbf{d}_v$	$\mathbb{R}^D \rightarrow \mathcal{S}^{B-1}$
Losses and objectives		
supervised loss (cross-entropy)	$\mathcal{L}_{\text{CE}}(\mathbf{y}, \mathbf{g}(\mathbf{x}; \mathbf{w}))$	$\mathbb{R}$
domain loss (DANN)	$\mathcal{L}_{\text{CE}}(b, \mathbf{d}_v(\mathbf{h}))$	$\mathbb{R}$
CORAL discrepancy	$\ \mathbf{C}_S - \mathbf{C}_T\ _F^2$	$\mathbb{R}_{\geq 0}$
BatchNorm (BN) parameters		
test-batch mean/var (per channel, batch β)	$\mu_{\text{BN}}, \sigma_{\text{BN}}$	$\mathbb{R}^D, \mathbb{R}_{>0}^D$
affine parameters	κ, λ	$\mathbb{R}^D, \mathbb{R}_{>0}^D$

B. Training details

In our experiments, we used either Nvidia A40 or A100 GPUs and all experiments use a single GPU. When reporting results, for the new experimental batch evaluation (experiment 5.1) and for the in-domain evaluation of source 3 (S3→S3 in Table 2), we perform 5-fold cross-validation and report the mean and standard deviation across folds. For the in-domain evaluation of source 8 (S8→S8), due to computational constraints, we perform re-runs in two different random splits.

For all methods, in Experiments 5.1 and 5.3 we set a maximum number of training epochs to 400 and implement an early stopping strategy, setting the patience to 80 epochs, so that if the validation loss does not decrease with respect to the current minimum loss for 80 epochs, the training is stopped. In Experiments 5.2 and 5.4, due to computational constraints, we set a maximum number of training epochs to 200 and a patience of 20. In all experiments, the checkpoint used for evaluating the test set is the one that achieved the lowest validation loss. By default, we use a ResNet50 as a feature encoder. As an optimizer, unless otherwise stated, we use Adam and the selected learning rate is decreased using a cosine annealing scheduler. The hyperparameters were manually tuned, and the exploration space for each method can be found in Supplementary Table A5

For the methods based on the CA-MAE foundation model, linear probing is performed, so that the weights of the feature encoder are frozen and only the classifier, which consists in one linear layer, is trained. The pretrained weights of this model are the weights of a ViT-S/16, provided in HuggingFace by Kraus et al. (2024), trained with the RxRx3 dataset and subsets of the JUMP-CP with samples treated for gene knockout and overexpression.

For the entropy-based methods TENT (Wang et al., 2021), DEYO (Lee et al., 2024) and ROID (Marsden et al., 2023), we consider every plate as a new domain, encoding one full plate at a time and performing 3 update steps per plate, with a learning rate of 0.001.

In the style transfer baseline, as the images were generated using a pre-trained diffusion model, the input for the feature extractor consisted in 3 channels instead of 5. Moreover, for computational efficiency, the images were generated with a resolution of 256x256.

For performing the TVN correction, control images from the corresponding matching plate of each perturbed sample are used for centering and whitening.

C. Dataset details

As explained in Section 5, the experiments were carried out on the JUMP-CP dataset, using source 3 and source 8. The classification task consists in classifying the MoA of the given image, similar to Haslum et al. (2023). As we need images that are correctly annotated, we use images that were treated with compounds that have a clear and well-defined MoA. Specifically, we use the images treated with the compounds used as positive controls, which are all present in every plate. The name of these compounds, along with their mechanism of action can be found in Supplementary Table A6. Source 3 contains 9 experimental batches and consists of 49,491 microscopy images (with 5 channels each) and 172 different plates. Source 8 contains 4 experimental batches and 56,160 microscopy images (with 5 channels each) and 195 different plates. For experiments with the CellProfiler features, we download the provided images from the CellPainting Gallery (Carpenter-Singh and Cimini Labs at the Broad Institute, 2026), and perform standardization by using the training features. We use these features as provided, and they were calculated on a well-level basis, so that this is the only baseline where accuracy is evaluated on a well level.

Preprocessing. As a preprocessing step, the images were downsized to a resolution of 512x512 and we performed illumination correction by clipping 0.01% of the brightest pixels. During training, the images were further cropped to 256x256 and random vertical and horizontal flipping was performed. At inference time, the center crop of the images with a resolution of 256x256 was used. Moreover, in all methods, except for the baseline with instance norm and the MAE-based methods, the images were normalized using the mean and standard deviation calculated for the training split. In the remaining methods, the images are preprocessed by normalizing the channel of each image individually, in order to match the pretraining procedure in the case of the CA-MAE baselines (Kraus et al., 2024).

For the TVN correction, additional negative controls, i.e. samples in which only DMSO was applied, were used. In total, the negative controls present in the 172 plates from the dataset described above, consist of 49,536 images.

Table A5. Considered hyperparameter space for all compared methods. The selected configurations for mechanism of action prediction based on manual search on validation set are shown in bold.

	Hyperparameter	Explored space
Baseline (ResNet)	Learning rate	{ 0.001 , 0.005}
	Scheduler	{ Cosine annealing , None}
	Batch size	{64, 128 }
Baseline (CA-MAE)	Learning rate	{0.001, 0.002, 0.003, 0.004, 0.005, 0.006, 0.007, 0.009 , 0.01}
	Batch size	{128}
DANN	Learning rate	{0.001}
	Batch size	{128}
	λ	{0.1, 0.5, 0.7, 2 , 3, 4}
	Domain classifier learning rate	{0.0001}
	Unlabeled batch size	{128}
CORAL	Learning rate	{0.001}
	Batch size	{128}
	γ	{ 0.05 , 0.2, 0.3, 1}
	Unlabeled batch size	{128}
CA-MAE (FM) + TVN	Learning rate	{ 0.001 , 0.002, 0.003, 0.004, 0.005, 0.006, 0.007, 0.009, 0.01}
	Batch size	{128}
Style-ID	Learning rate	{0.001}
	Batch size	{128}
ARM-BN	Learning rate	{0.001}
	Batch size	{8, 16, 32, 64, 128 }
CS-ARM-BN	Learning rate	{0.001}
	Batch size	{8, 16, 32, 64 }
	Negative controls batch size	{64, 128 }
ARM-CML	Learning rate	{0.001}
	Batch size	{128}
	Context channels	{12}
	Kernel size	{5}

Table A6. Compounds used as mechanism of action representatives for the classification task.

Compound name	Mechanism of action class
Aloxistatin	Cysteine protease inhibitor
FK-866	NAMPT inhibitor
AMG900	Aurora kinase inhibitor
Dexamethasone	Glucocorticoid agonist
LY2109761	TGF- β inhibitor
NVS-PAK1-1	PAK1 inhibitor
Quinidine	Sodium channel inhibitor
TC-S-7004	DYRK1A/B inhibitor

D. Additional compared methods

Domain-Adversarial Neural Networks (DANN) (Ganin and Lempitsky, 2014). DANN uses a domain discriminator $d_v : \mathbb{R}^D \rightarrow \mathcal{S}^{B-1}$ over features $\mathbf{a} = f_w(\mathbf{x})$ and uses gradient reversal to promote domain invariance:

$$\min_w \max_v \sum_{b=1}^B \sum_{(\mathbf{x}, \mathbf{y}) \in \mathcal{D}^{(b)}} \left[\mathcal{L}_{\text{CE}}(\mathbf{y}, g(\mathbf{x}, w)) + \lambda \mathcal{L}_{\text{CE}}(b, d_v(\mathbf{a})) \right], \quad (12)$$

where the second loss term is cross-entropy on domain labels and $\lambda > 0$ trades classification performance against domain objectives.

Correlation Alignment (CORAL) (Sun et al., 2015). Let $\mathbf{A}_S = \{\mathbf{a}_i\}$ and $\mathbf{A}_T = \{\mathbf{a}'_j\}$ be source and target features in a training batch; let $\mathbf{C}_S$ and $\mathbf{C}_T$ denote their covariance matrices. CORAL adds a moment-matching penalty to the supervised loss:

$$\min_w \sum_{b=1}^B \sum_{(\mathbf{x}, \mathbf{y}) \in \mathcal{D}^{(b)}} L_{\text{CE}}(\mathbf{y}, g(\mathbf{x}, w)) + \gamma \|\mathbf{C}_S - \mathbf{C}_T\|_F^2, \quad (13)$$

aligning second moments of source and target features during training. We trained CORAL on considering the samples in the training set to stem from the source distribution and the samples in the test set, from the target distribution.

Channel-Agnostic MAE (CA-MAE) + Typical Variation Normalization (TVN) (Kraus et al., 2024; Ando et al., 2017). As in the foundation model baseline, we use the embeddings from a frozen CA-MAE encoder, and perform a post-hoc correction with TVN. For TVN, given a set of batch- β control embeddings from the target batch, $\{\mathbf{z}_j^\beta\}$, $\boldsymbol{\mu}^{\text{ctrl}, \beta}$ and $\boldsymbol{\Sigma}^{\text{ctrl}, \beta}$ are estimated and used to apply a per-batch whitening:

$$\tilde{\mathbf{a}} = (\boldsymbol{\Sigma}^{\text{ctrl}, \beta})^{-\frac{1}{2}} (\mathbf{a} - \boldsymbol{\mu}^{\text{ctrl}, \beta}) \quad (14)$$

is performed. Then, the covariance is aligned to the covariance of the training data $\boldsymbol{\Sigma}^{\text{train}}$. The classifier c_θ is then trained with the resulting re-aligned representations $\tilde{\mathbf{a}}$. This matches first and second moments to the typical, i.e. control, distribution of each batch.

Generative methods: StyleID (Chung et al., 2024). We used the diffusion-based style transfer method proposed by Chung et al. (2024) to convert all images to the style of one specific experimental batch (Liu and Tuzel, 2016). Then, as in the supervised baselines, a ResNet50 and a linear classifier are trained to minimize the cross-entropy loss $\mathcal{L}_{\text{CE}}$ in the generated training samples.

DEYO (Lee et al., 2024). DeYO is a test-time adaptation method that extends entropy-minimization approaches by selecting and weighting target samples according to whether their predictions appear to rely on semantically meaningful structure rather than spurious cues. In addition to entropy minimization, DeYO defines a pseudo-label probability difference (PLPD) score by comparing the confidence of the pseudo-label before and after applying a transformation $\mathcal{T}$ that destroys the semantic meaning:

$$\text{PLPD}(\mathbf{x}) = (\hat{\mathbf{y}}_{\mathbf{x}} - \hat{\mathbf{y}}_{\mathcal{T}(\mathbf{x})})_p, \quad (15)$$

where $\hat{\mathbf{y}}_{\mathbf{x}}$ and $\hat{\mathbf{y}}_{\mathcal{T}(\mathbf{x})}$ are the model predictions of sample $\mathbf{x}$ and transformed sample $\mathcal{T}(\mathbf{x})$, respectively, and $p = \text{argmax}(\hat{\mathbf{y}}_{\mathbf{x}})$ is a pseudo-label estimated by the prediction of $\mathbf{x}$.

Samples with low entropy and high PLPD are selected and weighted more strongly during adaptation, so that the test-time objective can be written as:

$$\min_{\kappa, \lambda} \phi(\mathbf{x}) \mathbb{I}[\mathbf{x} \in \mathcal{S}] H(\hat{\mathbf{y}}), \quad (16)$$

where S denotes the set of selected target samples and $\phi(x)$ is a weighting function based on entropy and PLPD. The BatchNorm affine parameters are then updated by minimizing a weighted entropy over the selected target samples.

ROID (Marsden et al., 2023). ROID is an online test-time adaptation method based on weighted self-training. For an unlabeled target image x , ROID computes a certainty score from the prediction entropy and a diversity score relative to an exponential moving average of recent predictions, and combines them into a sample weight $w(x)$. These weights are then used in a self-training objective so that adaptation focuses on target samples that are both confident and diverse. To improve stability, ROID additionally averages the adapted parameters with the source parameters and estimates the target class prior to correct for label shift.

E. Additional experiments

E.1. Failure mode of BN adaptation — dependence on adaptation batch size

BN-based test-time adaptation implicitly relies on accurate estimation of target statistics. When only a few samples are available from a new batch (or when adaptation must be performed in small micro-batches due to memory constraints), these estimates become noisy. To stress-test this failure mode, we vary the number of target samples used to compute adaptation statistics (*adaptation batch size / context size* M) and evaluate methods that adapt BN parameters. Table A7 reports the results.

Previous BN-adaptation methods collapse for small M . Both AdaBN and TENT perform poorly when only very few target samples are available to estimate BN statistics (e.g., $M \in \{1, 2, 4\}$). Performance improves monotonically with M , approaching a plateau only once statistics can be reliably estimated (larger M). This explains why BN-based test-time adaptation can appear strong in benchmarks that provide a full target batch, yet fail in operational settings with limited or streaming access.

Control-stabilized context mitigates the small- M regime. Our control-stabilized variant (**CS-ARM-BN** in the table) remains strong even for extremely small numbers of perturbed samples. The key is that, while perturbed-class composition may be sparse, *negative controls are plentiful and consistently available in every batch*. Using controls to estimate batch statistics yields a stable estimate of batch-specific nuisance variation, making adaptation substantially less sensitive to the number of available perturbed samples.

Table A7. Comparison of MoA classification accuracy of different methods in a new experimental batch with different batch sizes. The accuracy values denote mean and standard deviation over 5-fold cross-validation, thus five training re-runs of each method.

Method	Labeled batch size								
	1	2	4	8	16	32	64	128	Full domain
ERM (no adaptation)	0.862 ± 0.060	0.862 ± 0.060	0.862 ± 0.060	0.862 ± 0.060	0.862 ± 0.060	0.862 ± 0.060	0.862 ± 0.060	0.862 ± 0.060	0.862 ± 0.060
TENT	0.126 ± 0.001	0.175 ± 0.013	0.230 ± 0.011	0.323 ± 0.012	0.550 ± 0.026	0.794 ± 0.021	0.881 ± 0.025	0.889 ± 0.031	0.908 ± 0.032
AdaBN	0.130 ± 0.005	0.447 ± 0.095	0.709 ± 0.052	0.839 ± 0.024	0.898 ± 0.014	0.919 ± 0.014	0.924 ± 0.014	0.927 ± 0.013	0.928 ± 0.013
ARM-BN	0.126 ± 0.007	0.514 ± 0.092	0.716 ± 0.052	0.832 ± 0.028	0.896 ± 0.019	0.923 ± 0.018	0.930 ± 0.018	0.933 ± 0.017	0.935 ± 0.018
CS-ARM-BN	0.923 ± 0.018	0.923 ± 0.018	0.924 ± 0.019	0.925 ± 0.018	0.927 ± 0.019	0.929 ± 0.019	0.930 ± 0.019	0.929 ± 0.020	0.923 ± 0.023

E.1.1. ABLATION STUDY. EFFECT OF THE NEGATIVE CONTROLS SET SAMPLE SIZE

In the previous experiment, we assume that other methods do not consider the presence of negative controls in biological data, so that the available samples are only perturbed ones, while in CS-ARM-BN we make use of negative controls, which results in a larger effective batch size. However, in this experiment we also study the effectiveness of standard ARM-BN in the presence of negative controls, so that we compare both methods using the same total batch size. In Table A8, we show that **CS-ARM-BN presents a higher accuracy** even when the **same amount of unlabeled data** is available, probably due to the fact that during the meta-learning phase, the model has already learned to normalize the intermediate features of the model using both negative controls and perturbed samples.

Table A8. Comparison between ARM-BN and CS-ARM-BN classification accuracy using the same number of samples in every batch, with the same proportion of negative controls per perturbed sample. The accuracy values denote mean and standard deviation over 5-fold cross-validation, thus five training re-runs of each method.

Method	Total batch size [perturbed : controls]				
	5 [1 : 4]	10 [2 : 8]	20 [4 : 16]	40 [8 : 32]	80 [16 : 64]
ARM-BN	0.799 $\pm$ 0.020	0.827 $\pm$ 0.024	0.846 $\pm$ 0.025	0.860 $\pm$ 0.026	0.870 $\pm$ 0.028
CS-ARM-BN (ours)	0.848 $\pm$ 0.035	0.875 $\pm$ 0.029	0.891 $\pm$ 0.028	0.899 $\pm$ 0.027	0.905 $\pm$ 0.026

E.2. Label shift experiment. Supplementary information.

E.2.1. EXPERIMENT DETAILS

In experiment 5.3 we show a comparison between methods under label shifts. To achieve batches that have a certain level of label shift, we sample a class probability vector from a Dirichlet distribution with a certain α parameter. Then we use this class probability vector to define a multinomial distribution and draw N samples from it, that we will use as input to our model. In total we use a labeled batch size of 36 in every method, and we include 288 negative samples in CS-ARM-BN.

One could argue that performance differences might be only due to a larger sample size, therefore, we show in Table A9 other baselines including the same amount of negative controls in each batch, as explained in the following section.

E.2.2. ABLATION STUDY. EFFECT OF THE NEGATIVE CONTROLS SET SAMPLE SIZE

In our method, CS-ARM-BN, negative controls are used both at training and at inference time, so the total batch size used to compute the BN parameters is larger than for the other methods. To demonstrate that the gains in performance in our method are not only due to a larger batch size but also due to the fact that the models were trained in a meta-learning fashion, we run the **AdaBN and ARM-BN** with the same amount of negative controls and **same total batch size as for CS-ARM-BN**. We don't perform this experiment for TENT because we argue that there is no reason why minimizing the entropy of the model outputs for control samples should be beneficial. In Table A9 we show that, even if including controls increases the performance of both methods, **CS-ARM-BN still shows the best performance**.

Table A9. Comparison of MoA classification accuracy of different methods in a new experimental batch under different degrees of label shift. A smaller α value corresponds to a larger degree of label shift.

Method	Label shift degree			
	$\alpha = 4$	$\alpha = 1$	$\alpha = 0.1$	$\alpha = 0.01$
ERM (no adaptation)	0.862 $\pm$ 0.060	0.862 $\pm$ 0.060	0.862 $\pm$ 0.061	0.862 $\pm$ 0.061
TENT	0.887 $\pm$ 0.014	0.832 $\pm$ 0.014	0.778 $\pm$ 0.016	0.156 $\pm$ 0.010
DEYO	0.887 $\pm$ 0.013	0.830 $\pm$ 0.014	0.773 $\pm$ 0.015	0.156 $\pm$ 0.010
ROID	0.894 $\pm$ 0.016	0.837 $\pm$ 0.017	0.792 $\pm$ 0.020	0.157 $\pm$ 0.011
Ada-BN	0.899 $\pm$ 0.012	0.851 $\pm$ 0.016	0.452 $\pm$ 0.010	0.229 $\pm$ 0.009
ARM-BN	0.894 $\pm$ 0.018	0.831 $\pm$ 0.017	0.416 $\pm$ 0.020	0.228 $\pm$ 0.009
Ada-BN (perturbed + controls)	0.879 $\pm$ 0.018	0.871 $\pm$ 0.019	0.815 $\pm$ 0.022	0.785 $\pm$ 0.033
ARM-BN (perturbed + controls)	0.861 $\pm$ 0.032	0.850 $\pm$ 0.033	0.785 $\pm$ 0.058	0.752 $\pm$ 0.068
CS-ARM-BN (ours)	0.927 $\pm$ 0.017	0.924 $\pm$ 0.018	0.906 $\pm$ 0.021	0.894 $\pm$ 0.022

E.3. Evaluation of CS-ARM-BN with a Vision Transformer-based architecture

Although, throughout the main manuscript, we have evaluated our method and the baselines using a ResNet50, in this section we also benchmark the best performing methods using LeViT (Graham et al., 2021), an architecture based on the Vision Transformer that employs Batch Normalization layers instead of the usual Layer Normalization. Particularly, we evaluate the methods on the most challenging task, under both label shift and strong domain shift. In Table A10, it can be seen that adapting the Batch Normalization layers and including negative controls in a meta-learning fashion is an effective strategy to mitigate domain and label shifts also when using a Vision Transformer architecture. We evaluated ERM, TENT, AdaBN and ARM-BN as in previous experiments, and additionally included two more recent TTA baselines, DEYO (Lee et al., 2024) and ROID (Marsden et al., 2023).

Table A10. Comparison of MoA classification accuracy of different methods across target sources under different degrees of label shift, using a **LeViT, a vision transformer-based architecture**. A lower α denotes a higher degree of label shift. **The accuracy values denote mean and standard deviation over re-runs with three different seeds.**

Method	S3 $\rightarrow$ S8		S8 $\rightarrow$ S3	
	$\alpha = 1$	$\alpha = 0.01$	$\alpha = 1$	$\alpha = 0.01$
(LeViT) no adaptation (BatchNorm)	0.607 $\pm$ 0.023	0.608 $\pm$ 0.019	0.536 $\pm$ 0.062	0.536 $\pm$ 0.051
(LeViT) TENT	0.675 $\pm$ 0.017	0.137 $\pm$ 0.002	0.640 $\pm$ 0.013	0.145 $\pm$ 0.005
(LeViT) DEYO	0.674 $\pm$ 0.014	0.137 $\pm$ 0.002	0.639 $\pm$ 0.015	0.145 $\pm$ 0.005
(LeViT) ROID	0.694 $\pm$ 0.018	0.138 $\pm$ 0.003	0.657 $\pm$ 0.015	0.147 $\pm$ 0.005
(LeViT) AdaBN	0.699 $\pm$ 0.015	0.165 $\pm$ 0.002	0.665 $\pm$ 0.014	0.168 $\pm$ 0.004
(LeViT) ARM-BN	0.735 $\pm$ 0.024	0.166 $\pm$ 0.003	0.702 $\pm$ 0.119	0.165 $\pm$ 0.001
(LeViT) CS-ARM-BN (ours)	0.791$\pm$0.025	0.707$\pm$0.021	0.722$\pm$0.015	0.610$\pm$0.017

E.4. Microscopy imaging data structure and domain alignment

Microscopy imaging data presents a hierarchical structure. In each imaging platform, also called *source* throughout this manuscript, reagents and cell cultures change over time and have to be replaced, which leads to different *experimental batches*. Moreover, within each one of these experimental batches, cells have to be laid out in what are known as *plates*, which are then placed under the microscope and images are obtained. As we have explained in Section 1, there are visible differences between plates, batches and sources. Moreover, each one of these plates has different positions, known as *wells*.

Given this hierarchy, a natural question is at which level to perform the domain alignment. While each source or batch has a larger number of samples, they are a set of other domains which might lead to a non-optimal alignment. In the following experiment, we show the adaptation results for AdaBN at the possible different levels: source, plate and batch, and we show that aligning at the plate level yields better results, even if fewer samples are available per domain.

Table A11. MoA classification accuracy of the AdaBN method adapting the BatchNorm statistics per source, per batch and per plate. Even if each plate consists of, naturally, fewer samples than each batch or source, the resulting accuracy is higher when the domain adaptation is performed per plate. Accuracy is an average over the S3 $\rightarrow$ S8 and S8 $\rightarrow$ S3 transfer experiments, shown in Table 2.

Alignment level	Accuracy	Number of domains	Samples per domain (avg.)
Source	0.773 $\pm$ 0.033	1	56,160
Batch	0.790 $\pm$ 0.014	4	14,040
Plate	0.798 $\pm$ 0.014	195	288

F. The in-context learning view on CS-ARM-BN

Our method CS-ARM-BN can be understood as a form of *in-context learning*: rather than retraining the model, the classifier conditions its predictions on a small *context* of unlabeled samples from the new experimental batch, without any gradient update. The negative controls play the role of a stable, class-agnostic reference that directly reflects the batch-specific technical variation.

As shown in Algorithm 1, this can be written in two equivalent ways: either as an explicit two-step procedure of calibration followed by inference (`model.adapt()`), or as a single context-conditioned forward pass (`model.cpredict()`), making the analogy to in-context learning explicit.

Algorithm 1 Pseudo code for CS-ARM-BN vs traditional image classifier pipelines | In-context view on CS-ARM-BN

```

### Standard Image Classifier Pipeline ###
# no calibration of classifier to data
x = load_data() # your biomedical images
model = load_model() # public classifier
pred = model.predict(x) # domain gap!

### CS-ARM-BN Classifier Pipeline ###
# adapt/calibrate classifier to your data
x = load_data() # your biomedical images
z = load_controls() # available controls
model = load_model() # public classifier
model.adapt(concat([x, z])) # calibrate
pred = model.predict(x) # neutralized gap

### In-context learning view on CS-ARM-BN
# adapt/calibrate classifier to your data
x = load_data() # your biomedical images
z = load_controls() # available controls
model = load_model() # public classifier
context = concat([x, z]) # assemble context
pred = model.cpredict(x, context=context)
    
```

G. Details on estimators

G.1. Setup

We model the BN activation mean for a sample x as:

$$\mu_{\text{obs},x} = \mu_{\text{domain}} + \mu_{\text{class}(x)} + \varepsilon_x, \quad (17)$$

where μ_{domain} is the batch-specific technical offset we wish to estimate, $\mu_{\text{class}(x)} = 0$ for control samples, $\mu_{\text{class}(x)} \neq 0$ for perturbed samples, and $\varepsilon_x \sim (0, \sigma^2)$ is additive noise.

We have C control samples and L perturbed samples, with $M = C + L$ total. The mean class effect across all perturbed samples is

$$\bar{\mu}_{\text{class}} = \frac{1}{L} \sum_{l=1}^L \mu_{\text{class}(l)}. \quad (18)$$

For any estimator $\hat{\mu}$, the MSE decomposes as:

$$\text{MSE}(\hat{\mu}) = \text{Bias}^2(\hat{\mu}) + \text{Var}(\hat{\mu}). \quad (19)$$

G.2. Estimator 1: ARM-BN and AdaBN (Perturbed Samples Only)

The estimator is the sample mean over the L perturbed samples:

$$\hat{\mu}^{\text{ARM-BN}} = \frac{1}{L} \sum_{l=1}^L \mu_{\text{obs},l} = \frac{1}{L} \sum_{l=1}^L (\mu_{\text{domain}} + \mu_{\text{class}(l)} + \varepsilon_l). \quad (20)$$

Bias.

$$\mathbb{E}[\hat{\mu}^{\text{ARM-BN}}] = \mu_{\text{domain}} + \bar{\mu}_{\text{class}}, \quad (21)$$

$$\text{Bias} = \mathbb{E}[\hat{\mu}^{\text{ARM-BN}}] - \mu_{\text{domain}} = \bar{\mu}_{\text{class}}. \quad (22)$$

The bias equals the mean class effect, which grows large under label shift.

Variance.

$$\text{Var}(\hat{\mu}^{\text{ARM-BN}}) = \text{Var}\left(\frac{1}{L} \sum_{l=1}^L \varepsilon_l\right) = \frac{\sigma^2}{L}. \quad (23)$$

MSE.

$$\text{MSE}(\hat{\mu}^{\text{ARM-BN}}) = \bar{\mu}_{\text{class}}^2 + \frac{\sigma^2}{L}. \quad (24)$$

G.3. Estimator 2: Controls Only

The estimator is the sample mean over the C control samples. Since controls satisfy $\mu_{\text{class}} = 0$:

$$\hat{\mu}^{\text{ctrl}} = \frac{1}{C} \sum_{c=1}^C \mu_{\text{obs},c} = \frac{1}{C} \sum_{c=1}^C (\mu_{\text{domain}} + \varepsilon_c). \quad (25)$$

Bias.

$$\mathbb{E}[\hat{\mu}^{\text{ctrl}}] = \mu_{\text{domain}}, \quad (26)$$

$$\text{Bias} = 0. \quad (27)$$

The estimator is unbiased because controls carry no class-specific signal by definition.

Variance.

$$\text{Var}(\hat{\mu}^{\text{ctrl}}) = \frac{\sigma^2}{C}. \quad (28)$$

This is large when C is small, which is the typical regime in drug-screening experiments.

MSE.

$$\text{MSE}(\hat{\mu}^{\text{ctrl}}) = 0 + \frac{\sigma^2}{C}. \quad (29)$$

G.4. Estimator 3: CS-ARM-BN (Controls + Perturbed Samples)

The estimator is the sample mean over all $M = C + L$ samples:

$$\hat{\mu}^{\text{CS-ARM-BN}} = \frac{1}{M} \sum_{u \in \mathcal{C}^\beta} \mu_{\text{obs},u} = \frac{1}{M} \left(\sum_{c=1}^C \mu_{\text{obs},c} + \sum_{l=1}^L \mu_{\text{obs},l} \right). \quad (30)$$

Expanding each term:

$$\begin{aligned}
 \hat{\mu}^{\text{CS-ARM-BN}} &= \frac{1}{M} \left(\sum_{c=1}^C (\mu_{\text{domain}} + \varepsilon_c) + \sum_{l=1}^L (\mu_{\text{domain}} + \mu_{\text{class}(l)} + \varepsilon_l) \right) \\
 &= \mu_{\text{domain}} + \frac{1}{M} \sum_{l=1}^L \mu_{\text{class}(l)} + \frac{1}{M} \sum_u \varepsilon_u \\
 &= \mu_{\text{domain}} + \frac{L}{M} \bar{\mu}_{\text{class}} + \frac{1}{M} \sum_u \varepsilon_u.
 \end{aligned} \tag{31}$$

Bias.

$$\mathbb{E}[\hat{\mu}^{\text{CS-ARM-BN}}] = \mu_{\text{domain}} + \frac{L}{M} \bar{\mu}_{\text{class}}, \tag{32}$$

$$\text{Bias} = \frac{L}{M} \bar{\mu}_{\text{class}}. \tag{33}$$

The bias is attenuated by the factor $L/M < 1$ relative to Estimator 1. The more controls are included (larger C , smaller L/M), the smaller the bias.

Variance.

$$\text{Var}(\hat{\mu}^{\text{CS-ARM-BN}}) = \text{Var}\left(\frac{1}{M} \sum_u \varepsilon_u\right) = \frac{\sigma^2}{M}. \tag{34}$$

Since $M > L$ and $M > C$, this variance is strictly smaller than those of Estimators 1 and 2.

MSE.

$$\text{MSE}(\hat{\mu}^{\text{CS-ARM-BN}}) = \left(\frac{L}{M}\right)^2 \bar{\mu}_{\text{class}}^2 + \frac{\sigma^2}{M}. \tag{35}$$

G.5. Comparison

Table A12. MSE decomposition for the three estimators of μ_{domain} .

Estimator	Bias ²	Variance	Notes
ARM-BN and AdaBN	$\bar{\mu}_{\text{class}}^2$	σ^2/L	Large bias under label shift
Controls-only	0	σ^2/C	Large variance when C is small
CS-ARM-BN	$(L/M)^2 \bar{\mu}_{\text{class}}^2$	σ^2/M	Trade-off between the two above

CS-ARM-BN achieves the lowest MSE by trading a residual bias shrunk by $(L/M)^2$ relative to ARM-BN against the lowest variance of all three estimators (since $M \geq \max(L, C)$). This bias-variance trade-off is most favourable in the typical drug-screening regime: few controls (C small) and moderate-to-severe label shift.

H. Detecting edge cases with uncertainty

To further analyze whether the model can identify atypical samples at inference time, we performed an additional experiment based on predictive uncertainty. In our setup, the cross-validation folds and repeated training runs naturally provide a deep ensemble, which we use to estimate uncertainty for each test image. Specifically, we compute the entropy of the mean predicted class probabilities across all ensemble members. We then inspect samples with different uncertainty levels to determine whether highly uncertain predictions correspond to potential edge cases. As shown in Figure A3, most images exhibit very low uncertainty values (approximately 10^{-6} to 10^{-3}), indicating stable predictions. In contrast, a subset of samples shows substantially higher uncertainty, reaching up to approximately 10^{-1} . Visual inspection reveals that these samples often contain unusual technical artifacts, such as extreme high-intensity pixels in the green channel, suggesting that uncertainty can serve as a useful signal for detecting sample-specific, non-biological artifacts that are not well represented in the training data.

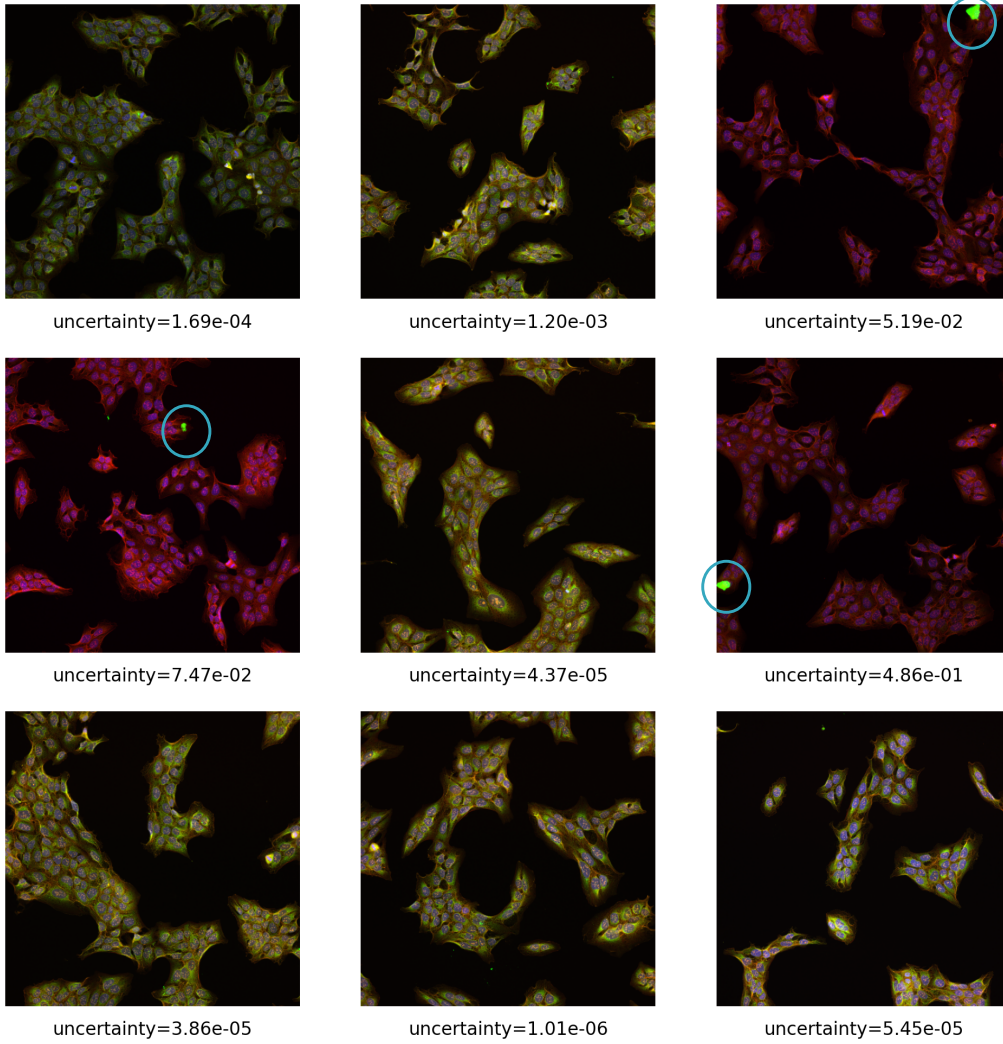
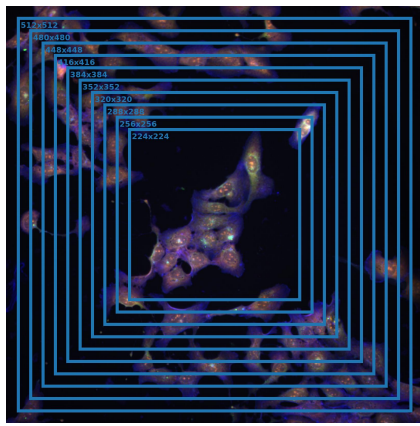


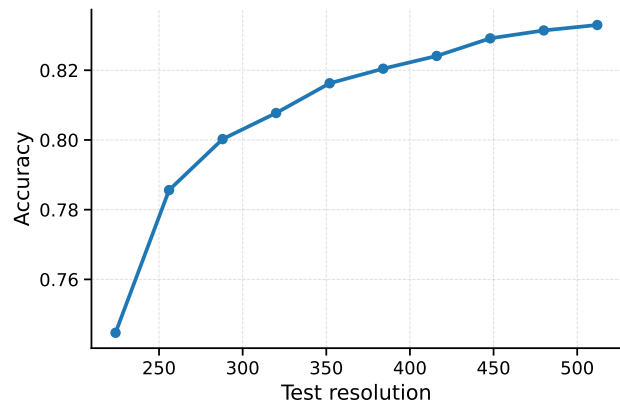
Figure A3. Representative examples of model predictions with corresponding deep ensemble uncertainty scores. Most samples exhibit low uncertainty (from approximately 1×10^{-6} to 1×10^{-3}), reflecting stable predictions. In contrast, a subset of images shows markedly higher uncertainty (up to 4.86×10^{-1}), often associated with atypical visual artifacts such as localized high-intensity pixels in the green channel (highlighted with blue circles). These results suggest that such artifacts correspond to out-of-distribution, sample-specific effects that increase uncertainty.

I. Test-Time Scaling

In this section, we evaluate the effect of varying image resolution at inference time while keeping the trained model fixed. For computational efficiency, in the experiments reported in the main paper, we use a center crop of each image at inference time, as described in Section C. However, this protocol may discard useful visual information that is present outside the cropped region or only becomes available when the image is processed at a higher spatial resolution. As shown in Figure A4, increasing the input resolution at test time leads to improved performance, even though the model itself is not retrained under this modified setting. This suggests that the learned representations can benefit from higher resolution at inference time. These results highlight a trade-off between computational cost and predictive performance, and indicate that further gains may be obtained by using higher-resolution inputs when inference-time efficiency is less constrained.



(a)



(b)

Figure A4. Test-time scaling for MoA prediction. (a) Example microscopy image center-cropped at increasing input resolutions, illustrating how higher-resolution crops retain additional spatial context and visual detail. (b) MoA classification accuracy as a function of test-time input resolution, with the trained model kept fixed. Accuracy improves as the inference resolution increases, suggesting that the learned representations can benefit from higher-resolution inputs without retraining.